Direct Manipulation and Natural Language Programming, Together at Last?

PARKER ZIEGLER, University of California, Berkeley, USA
DAVID MINH-DUY CAO, University of California, Berkeley, USA
JUSTIN LUBIN, University of California, Berkeley, USA
SARAH E. CHASINS, University of California, Berkeley, USA

Decades of programming languages research has contributed novel approaches to program editing that go beyond modifying text, including direct manipulation programming, structure editing, and automated refactoring tools. However, the rapid growth of natural language programming largely reinforces a view of programs as text and program editing as (unstructured) text transformation. How can we develop unified programming systems that bridge the gap between these approaches, supporting multiple editing paradigms in concert? And how would such systems change the way we program? We take a first step toward answering these questions by introducing a framework that enables program editing via both direct manipulation and natural language, and instantiate this framework in a variant of the `cartokit` direct manipulation programming system ($\texttt{cartokit}_{\text{DM+NL}}$). Our key insight is to treat programs as sequences of *structured edits* and to use an *edit language* as a shared interface for both direct manipulation and natural language interactions, leveraging constrained decoding to support the latter. Using our instantiation, we conducted a within-subjects study (N=18) to understand how the *combination* of direct manipulation and natural language as editing modalities changes the programming process compared to each modality alone. Perhaps surprisingly, we found that study participants overwhelmingly favored direct manipulation editing, performing just 6.14% of edits via natural language. Our thematic analysis of study sessions revealed that direct manipulation aided task decomposition, encouraged incremental editing, and helped mitigate known challenges in natural language programming related to understanding model capabilities and interpreting model-generated code. Conversely, natural language editing came into play largely to automate, parameterize, and replay known edits that would otherwise be repeated tediously by hand. Our edit-based framework and study findings lay out a possible pathway for future research on programming systems that blend natural language with alternative editing modalities, building on the foundation of edit languages.

CCS Concepts: • **Human-centered computing** → **Empirical studies in HCI**; **User interface programming**; • **Software and its engineering** → **Graphical user interface languages**.

Additional Key Words and Phrases: direct manipulation programming, natural language programming, edit languages, structured edit sequences, cartokit, geospatial data

1 Introduction

The programming languages community has devoted decades to developing new approaches to program editing beyond text-based manipulation, from direct manipulation programming [19, 35, 36, 93] to structure editing [1, 53, 60, 78] to automated code refactoring in IDEs [51, 61, 71, 79]. At the heart of these techniques is a perspective that "programs are not [just] text" [78]: they possess *structure* that programming systems can (and should!) exploit to assist in the editing process. Conversely, the recent arrival of code-generating large language models (LLMs) has largely reinforced a view of programs as text, and of program editing as a process of (unstructured) text transformation. LLMs generate code in a purely probabilistic fashion, sampling new tokens from a distribution conditioned on the previously generated token sequence. Recent work has attempted to reintroduce some degree of structure in this process by better *contextualizing* language model

Authors' Contact Information: Parker Ziegler, peziegler@cs.berkeley.edu, University of California, Berkeley, Berkeley, California, USA; David Minh-Duy Cao, dmcao@berkeley.edu, University of California, Berkeley, Berkeley, California, USA; Justin Lubin, justinlubin@berkeley.edu, University of California, Berkeley, Berkeley, California, USA; Sarah E. Chasins, schasins@cs.berkeley.edu, University of California, Berkeley, Berkeley, California, USA.

code completion with program information from language servers [11] and *constraining* language model code generation with semantic and type constraints [54, 56, 86]. Still, it remains unclear how we can integrate the potential of natural language programming and LLMs with the rich research in our community on programming paradigms that operate beyond text.

A promising approach, which we take in this paper, is to model programs not as arbitrary sequences of tokens or expressions, but as *sequences of structured edits* applied to some starting expression e_0:

$$e = (\delta_n \circ \cdots \circ \delta_1)\, e_0$$

This idea is already manifest in *edit languages*—such as Omar et al. [60]'s "actions" in Hazel, Meng et al. [51]'s "edit operations" in Sydit, Ziegler et al. [96]'s "`diffs`" in `cartokit`, or Petricek and Edwards [65]' "document edit types" in Denicek—that give precise syntax and semantics to edits, defining how programs (and their outputs) evolve at a fine level of granularity.

Given a structured language of edits, the next question is how to generate them. We can imagine different possibilities across diverse program editing contexts. In a structure editor like Hazel [59, 60], programmer interactions at the cursor dispatch edit actions that modify the "edit state" (i.e., the AST) directly according to Hazelnut's action semantics. In direct manipulation programming systems like Sketch-n-Sketch [19, 35], FuseDM [93], and `cartokit` [96], GUI interactions (e.g., button presses, slider drags, mouses presses) generate edits, which are subsequently consumed by program modification techniques (e.g., fusion [93], `patch-recon` [96]) that translate these edits into transformations on the program and its output. In this paper, we explore how we can extend the notion of edit sequence generation to natural language programming, using a large language model (LLM) to produce *structured* program edits (e.g., "actions," "edit operations," "diffs")—as opposed to *unstructured* program text—from programmers' prompts.

Bringing Together Direct Manipulation and Natural Language Programming. Unifying multiple edit modalities around an edit language as a shared interface opens up new directions for programming systems that support interachangeable editing between paradigms. A programming system supporting multiple edit modalities also lets us explore open-ended research questions about *how the programming process changes* when paradigms are brought together.

In this paper, we take a first step toward this vision by specifically considering two programming paradigms, direct manipulation programming and natural language programming, asking: *How does the programming process change when both direct manipulation and natural language are available as editing modalities in a single programming system*? To explore this question, we first extend the `cartokit` direct manipulation programming system for geospatial visualization [96] with support for natural language editing in a variant we call $\texttt{cartokit}_{\text{DM+NL}}$, using `cartokit`'s existing structured `diffs` as a shared edit language. In our variant, programmers' natural language prompts are translated by an LLM to `diff` sequences, which can be directly interpreted by `cartokit`'s existing `diff` semantics for GUI interactions. For example, a prompt like "Visualize the Democratic vote share by precinct of the 2024 election data layer, using a political color scheme" might produce the following `diff` sequence, which will modify the marks for a particular map layer and select a data column to control the fill color of layer features:

```
  setChannel("election__2024", "scheme",         (d) → "schemeBlues")
∘ setChannel("election__2024", "fill-attribute", (d) → d["pct_dem_lead"])
∘ setChannel("election__2024", "layer-type",     (d) → "Choropleth")
```

We use constrained decoding [27, 38, 64] to enforce that LLM-generated `diff` sequences are both syntactically valid and satisfy additional semantic constraints (e.g., that numeric data associated with `diffs` fall within a certain interval, or that `diffs` only reference existing map layers and column

names). LLM-generated `diffs` integrate directly with `cartokit`'s `patch-recon` architecture; from the system's perspective, they appear as if they had been generated by GUI interactions. We then conducted a within-subjects user study (N=18) in which participants used (i) our new unified direct manipulation and natural language (DM+NL) programming system, (ii) a natural language (NL) programming system (GitHub Copilot [28] with GPT-5 [63]), and (iii) a direct manipulation (DM) programming system (`cartokit` [96]) to author JavaScript programs for interactive maps.

Key Findings. Our study revealed new insights on how programmers move and choose between direct manipulation and natural language editing when both are available. Surprisingly, participants overwhelmingly favored direct manipulation, performing just 6.17% of all edits through natural language prompting. We observed that direct manipulation functionality scaffolded the programming process, guiding participants from higher-level edits to lower-level refinements and shifting the main editing challenge from recall (*How do I describe my intended edit?*) to recognition (*What part of the GUI can help me make my edit?*). We also found that direct manipulation encouraged incremental editing that helped participants assess correctness and bootstrapped program understanding. When participants did use natural language, it was primarily to scale up repetitive edits or to obtain guidance when they did not know how to achieve the edit via direct manipulation. Interestingly, direct manipulation helped mitigate key challenges in natural language programming by shaping how participants phrased prompts and providing alternative views to inspect model-generated edits. Cumulatively, our findings lay the foundation for understanding programming systems that blend these two editing paradigms, building on the foundation of edit languages.

Contributions. In summary, we make the following contributions:

- Quantitative results and a thematic analysis from a within-subjects user study exploring unified direct manipulation and natural language (DM+NL) programming, with comparisons to direct manipulation (DM) programming and natural language (NL) programming. To our knowledge, this is the first user study of DM+NL programming as well as the the first user study of DM programming.
- To enable the above, a framework for blending direct manipulation and natural language programming, building on the idea of structured edit languages (e.g., `diffs`).
- An instantiation of this framework in $\texttt{cartokit}_{\text{DM+NL}}$, a programming system that supports authoring JavaScript programs for geospatial visualization through both direct manipulation and natural language. This system is, to our knowledge, the first *programming system* to support both edit modalities.

2 Background

Program edits, patching, and reconciliation. Ziegler et al. [96] introduced `patch-recon`, an architecture for synchronizing programs and outputs in a direct manipulation programming system. The framework specifically handles cases where a user has triggered an update via a direct manipulation interaction (e.g., a button press, slider drag, mouse press), and the system must modify both the program and output in response to this interaction. In a `patch-recon` system, each direct manipulation interaction produces a *structured* program edit, called a `diff`. Then, a system developer specifies two complementary functions that operate over `diffs`:

(1) A syntactic diffing operation, `patch` ($\text{Diffs}_{\mathcal{L}} \times \text{Prog}_{\mathcal{L}} \rightarrow \text{Prog}_{\mathcal{L}}$), that takes in a `diff` δ and a current program P for some language $\mathcal{L}$ and *incrementally* updates P to P' based on δ.
(2) A reconciliation function, `recon` ($\text{Diffs}_{\mathcal{L}} \times \text{Val}_{\mathcal{L}} \rightarrow \text{Val}_{\mathcal{L}}$), that takes in a `diff` δ and a current program output V and *incrementally* updates V to V' based on δ.

Finally, by proving a *correspondence* between `patch` and `recon` with respect to all `diffs`, a system developer can leverage `patch-recon` for parallel, incremental updates of the program and output. Informally, proving patch-reconciliation correspondence involves demonstrating that `recon`'s semantic update of the output effectively emulates `patch`'s syntactic update of the program.

The primary advantages of `patch-recon` center on performance and correctness. For long-running programs, `patch-recon` produces faster program output updates than naive whole-program re-evaluation, with speedups increasing as program run time grows. Proving patch-reconciliation correspondence also guarantees that the current program always evaluates to the current output without actually requiring whole-program re-evaluation; in other words, all edits keep the program and output in sync. For more details, we refer readers to Ziegler et al. [96].

What is in a `diff`*?* Each `diff` represents a structured program edit dispatched by a direct manipulation interaction. `patch-recon` is not prescriptive about *what* information should be encoded in a `diff` or how. For example, for a given application, `diffs` could reference tree edit operations on an AST [1, 60], user actions in a record-and-replay system [17], operations in op-based CRDTs [75], edits produced by the standard POSIX `diff` command [30], or any other structured information describing program edits. `diffs` are defined by the system developer and should encode information necessary to design `patch` and `recon` functions and for proving patch-reconciliation correspondence. For this work (and our implementation, §4), we reuse the instantiations of `diffs`, `patch`, and `recon` in `cartokit` described in §4 of Ziegler et al. [96] ($\texttt{diff}_{\mathcal{L}_{ck}}$, $\texttt{patch}_{\mathcal{L}_{ck}}$, $\texttt{recon}_{\mathcal{L}_{ck}}$) with modest extensions. We detail extensions specific to this work, including both new `diffs` and extensions to the proof of `patch-recon` correspondence, in Appendix B.

Constrained Decoding. Constrained decoding (CD) [27, 38, 54, 56] is a technique that enforces constraints on the sequence of tokens generated by a language model. CD works by modifying a language model's sampling procedure at inference time, preventing token selection that would violate the constraints. More concretely, CD uses a *completion analysis* [56] or *completion engine* [54, 86] to check if a selected token could lead to a valid completion of the current token sequence and, if not, backtracks to select a different token.

3 Unifying Direct Manipulation and Natural Language via Program Edits

Given a direct manipulation programming system with a definition of `diffs`, `patch`, and `recon`, as well as a proof of patch-reconciliation correspondence, we can now extend the system to support program editing via natural language. We highlight that the extension is straightforward: rather than using an LLM to generate new (unstructured) program text from a user's natural language prompt, we instead generate *sequences of (structured)* `diffs`. We use constrained decoding to enforce that the LLM produces only valid `diffs` according to constraints Φ provided by the system developer. More formally:

$$cd(\mathsf{llm}, \mathsf{prompt}, \Phi) = (\delta_n \circ \cdots \circ \delta_1) \quad \text{such that} \quad \forall\, i.\ \delta_i \in \mathsf{Diffs}_{\mathcal{L}}$$

The key insight in this approach is to leverage `diffs` as a *shared language of program edits* that programming tools can generate either from direct manipulation edits or from natural language prompts (Figure 1). For a system developer, the primary additional implementation burden is to supply Φ, especially to provide the context-free grammar of allowable `diffs`. Some constrained decoders also support specifying additional semantic or type constraints by annotating a context-free grammar (e.g., [56, 64]). Because our framework is agnostic to the choice of constrained decoder, system developers may use these tools when syntactic constraints are insufficient for modeling their definition of $\mathsf{Diff}_{\mathcal{L}}$.

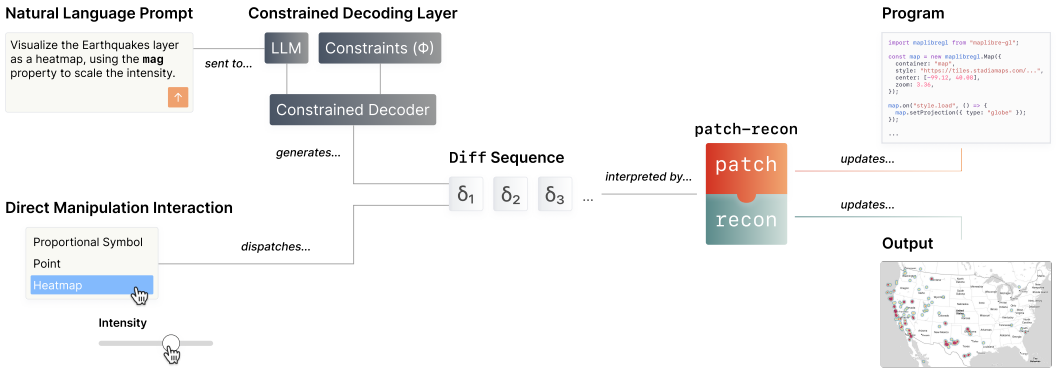


Fig. 1. **Generating program edits (`diffs`) using direct manipulation and natural language.** To generate valid structured `diffs` from natural language prompts, we can use constrained decoding, which enforces that an LLM produces outputs guaranteed to satisfy the provided constraints Φ on `diffs`. With this addition, a `patch-recon` system can use *both* natural language and direct manipulation edits to produce `diffs`; after a `diff` is generated via either a natural language interaction or a direct manipulation interaction, the system applies both `patch` and `recon` to incrementally update the program and output, respectively. This framework is agnostic to the design of `diffs`, choice of model, choice of constraints, and choice of constrained decoder.

This approach comes with several key advantages. First, because natural language prompts produce `diffs`, natural language edits enjoy the same performance and correctness benefits as direct manipulation edits in `patch-recon`—that is, the system can apply `diffs` incrementally, and `diff` application produces provably correct, parallel updates to the program and output. (Although note that model latency affects the overall performance of `diffs` generated via natural language relative to those generated via direct manipulation, which we discuss in §7.2.) Second, natural language edits—again by virtue of producing `diffs`—inherit some programming system conveniences previously only associated with direct manipulation. For example, natural language edits can be tracked in an edit history and (incrementally) undone or redone, and their effects on interface state are immediately visible to the programmer. As we discuss in §7.3, this aspect of our design mitigates challenges in understanding LLM capabilities and interpreting model-generated code that occur in natural language programming settings [46, 57, 94]. Finally, constraining the LLM to generate *structured edits* in lieu of full programs or textual edits may offer benefits for reliability. For example, LLM-backed tools can struggle to adhere to semantic constraints on programs because they generate program text probabilistically [21, 54]. By limiting generation to a fixed space of edits that enforce those semantic constraints, our framework sidesteps this problem.

4 Implementation

We extended `cartokit`, a direct manipulation programming system for geospatial visualization, with support for natural language editing in a variant we call $\texttt{cartokit}_{\text{DM+NL}}$. Our implementation adds only 780 lines of code (LOC) to `cartokit`, which was a ≈13,000 LOC codebase at the time of modification. 625 LOC are TypeScript changes implementing our constrained decoding approach using Structured Outputs in the OpenAI API [64] with GPT-5 as the underlying model [63]. Concretely, this involved encoding each of `cartokit`'s `diffs` as JSON Schema [73] using the Zod schema validation library [50]. We formalized both the context-free grammar describing `cartokit`'s `diffs` as well as additional constraints on the data associated with `diffs`. As an example of the additional constraints, some `diffs` in $\texttt{cartokit}_{\text{DM+NL}}$ support modifying `fill` and `stroke` colors

of map features. To constrain GPT-5 to generating only valid 6-character string hex codes for these `diff`s, one can use the following Zod encoding: `z.string().regex(/^#[0-9A-Fa-f]{6}$/)`. We provide additional details of our `diff`-to-Zod encoding in Appendix A. The remaining 155 lines of additional code implement Svelte GUI components [32] for entering prompts and displaying warnings and errors based on model responses.

Critically, our implementation of $\texttt{cartokit}_{\text{DM+NL}}$ allows full reuse of `cartokit`'s existing `patch-recon` implementation; `diff` sequences generated with GPT-5 are processed in the same manner as `diff`s dispatched by direct manipulation interactions. This also means that natural language edits have the same guarantees around efficient incremental updates, deterministic JavaScript code generation, and the ability to be incrementally undone or redone.

5 Experiment Design

Having developed $\texttt{cartokit}_{\text{DM+NL}}$, we set out to understand how the *combination* of direct manipulation and natural language as a programming paradigm changes the programming process compared to direct manipulation or natural language alone. Concretely, we asked the following research questions:

- **RQ1** Given the task of reaching a fixed target program, how does the *combination* of direct manipulation and natural language as editing modalities affect participants' success rate or completion time compared to direct manipulation or natural language alone?
- **RQ2** How does the *combination* of direct manipulation and natural language as editing modalities influence how participants edit programs, write prompts, and assess program correctness relative to direct manipulation or natural language alone?
  - ↪ **RQ2a** Do we observe differences in participants' approaches based on prior programming experience?

We addressed these questions through a within-subjects lab study with N=18 geospatial data users programming interactive maps using three tools: $\texttt{cartokit}_{\text{DM+NL}}$ (DM+NL), `cartokit` (DM only), and GitHub Copilot (NL only). We evaluated **RQ1** quantitatively by measuring time-on-task and task completion, and by analyzing telemetry data collected in each study condition. We assessed **RQ2** and **RQ2a** qualitatively through an inductive, reflexive thematic analysis [13] of study sessions.

5.1 Participants

We conducted live study sessions with 18 (one transmasculine, five female, 10 male, two non-disclosed) participants, the majority of whom ($66.\overline{6}$%) were professional data journalists. We focused on data journalists as a study population for two reasons. First, many data journalists interact regularly with geospatial data and produce interactive maps as part of their work, making them target users for programming systems that can assist in this process. Second, many data journalists come from "non-traditional" computing backgrounds—that is, they may not have developed programming experience through formal computer science education. Given that direct manipulation and natural language programming systems are often designed specifically to support such users, data journalists are a relevant community for understanding how these paradigms (and their combination) affect the programming process. We supplemented our participant pool with students and researchers in journalism, Earth sciences, and geography from multiple research universities in the United States. We recruited participants through the first author's professional networks, a Slack workspace for data journalists, and university mailing lists. Table 1 provides more detail on each participant's background.

To help answer **RQ2a**, we used a screening survey to intentionally recruit an equal number of expert and near-novice programmers based on self-reported experience and background with

Table 1. **Backgrounds of study participants.** The "NN" prefix denotes near-novice participants (left) and the "E" prefix denotes expert participants (right). For student participants, letters in parentheses indicate areas of expertise (J ↦ Journalism, ES ↦ Earth Sciences, G ↦ Geography, CS ↦ Computer Science).

ID	JS Exp. (Yrs.)	JS Map. Exp. (Yrs.)	Occupation	ID	JS Exp. (Yrs.)	JS Map. Exp. (Yrs.)	Occupation
PNN1	1	<1	Data Journalist	PE1	5	4	Data Journalist
PNN2	<1	<1	Grad. Student (J)	PE2	10	5	Data Journalist
PNN3	0	0	Data Journalist	PE3	14	14	Data Journalist
PNN4	4	0	Grad. Student (ES)	PE4	15	13	Data Journalist
PNN5	2	0	Postdoc. (G)	PE5	3	2	Data Journalist
PNN6	2	0	Data Journalist	PE6	4	3	Data Journalist
PNN7	<1	<1	Undergrad. Stu. (CS, G)	PE7	15	13	Data Journalist
PNN8	<1	0	Grad. Student (J)	PE8	7	7	Data Journalist
PNN9	<1	<1	Grad. Student (J)	PE9	5	5	Data Journalist

JavaScript programming and JavaScript mapping libraries. We considered participants with at least three years of JavaScript programming experience and at least two years of programming experience with JavaScript mapping libraries to be experts. Conversely, near-novices self-reported between zero and two years of programming experience with JavaScript and less than one year of programming experience with JavaScript mapping libraries. PNN4 was the only participant that straddled these classification criteria. Despite having JavaScript experience, they noted that their use of JavaScript was very sporadic. Given this fact—in concert with their zero years of experience with JavaScript mapping libraries—we chose to include them in the near-novice group.

Domain Expertise. Importantly, our near-novice categorization is not indicative of participants' experience working with geospatial data or their background in cartography. In fact, many of the near-novice programmers in our participant pool are expert cartographers with deep expertise in geographic information systems (GISs), geospatial analysis, and cartographic design; they simply do not currently use JavaScript to do this work. However, the goal of this experiment is to understand direct manipulation and natural language as paradigms for *programming* systems, so our expert vs. near-novice labels refer only to programming expertise.

5.2 Study Conditions and Protocol

Our within-subject study included three conditions:

(1) **DM+NL Condition**: $\texttt{cartokit}_{\text{DM+NL}}$.
(2) **DM Condition**: `cartokit` [96].
(3) **NL Condition**: GitHub Copilot with GPT-5. We gave participants access to GitHub Copilot's web interface [28] using GPT-5 [63] as the underlying model.

Within a given condition, each participant completed the following procedure.

Tutorial (5 min). Before starting the tasks, the participant read a short tutorial highlighting key features of the interface they would be using. Tutorials were not step-by-step guides or walkthroughs of example scenarios; instead, tutorials focused on how to access various features of each interface. We provide all three tutorials in the supplementary material.

Task A, Reproduction (30 min). In Task A, we asked participants to develop a JavaScript program to reproduce a specific map published in The Washington Post. With three conditions, each

participant attempted to reproduce three different target maps—one per condition—over the course of the study. We assigned map-condition pairings in counterbalanced order to control for learning effects; see *Ordering* below. We provided participants with a static image of the target map, the URL of the news article containing the map, light instructions about the features of the map to reproduce, and the URL of the GeoJSON [16] data to use for the task. Before conducting the study, we tested whether using the full task description text as a prompt in the two natural language conditions was sufficient for completing the overall task; in our testing, it was not. All GeoJSON datasets were stored in a public Cloudflare R2 bucket and available for download to the participants' machines. To complete the task in every condition, we required participants to execute code in StackBlitz [40], an online JavaScript IDE. We added this requirement to ensure that generated code (i) executed without syntactic or semantic errors and (ii) produced an output map meeting the minimum criteria for reproduction according to our rubric. Participants could abandon the task at any time.

Task B, Exploration (10 min). In Task B, we asked participants to explore two geospatial datasets with the goal of authoring a JavaScript program that produced a map they considered "desirable." As in Task A, we provided participants with URLs for the two GeoJSON datasets, assigned in counterbalanced order; participants could choose to use one or both in their final map. Participants could end the task early by declaring they had reached a desirable map or by abandoning the task. Note that there are no quantitative measures associated with Task B. This task augments the qualitative portion of our data analysis, offering a portion of the session where we can observe how the programming paradigms affect how participants engage in exploratory programming work, without the constraint of a researcher-assigned target output.

Ordering. Task-condition pairings were assigned using a counterbalanced Latin square. With three conditions, we have six possible orderings of conditions. Thus, with 18 participants, each distinct ordering was completed by a group of three participants.

NASA-TLX. After completing each task, participants completed the NASA Task Load Index (NASA-TLX) [34] to report perceived workload for each condition-task pairing. Following guidance and findings from prior work [15, 37], and to reduce burden on participants, we chose to remove subscale weighting that requires participants to perform many pairwise comparisons on the perceived importance of each subscale. Instead, each participant indicated perceived workload along each subscale, and we take the unweighted mean of these values to estimate overall workload. This variation is sometimes referred to as Raw TLX (RTLX) [33].

Post-Interview and Survey (10-15 min). After completing the two tasks in each condition (2 Tasks / Condition × 3 Condition = 6 Tasks), participants reflected on their experiences in each condition in a semi-structured interview. Discussion centered on how each condition directed participants' attention while programming, how participants assessed program correctness, strategies for prompting, and strategies for navigating each interface, among other behaviors.

Session Duration and Compensation. Participants took part in the study remotely over Zoom over the course of 2-2.5 hours. Each participant shared their screen with the experimenter during the session, and we recorded both their screen and audio. In addition, we instrumented $\texttt{cartokit}_{\text{DM+NL}}$ to record anonymized `diff` sequences triggered by direct manipulation, as well as prompts and `diff` sequences triggered by natural language edits. Likewise, we instrumented `cartokit` to record anonymized `diff` sequences capturing direct manipulation interactions. Participants received compensation in the form of a $50 gift card or a $50 donation to a 501(c)(3) of their choice.

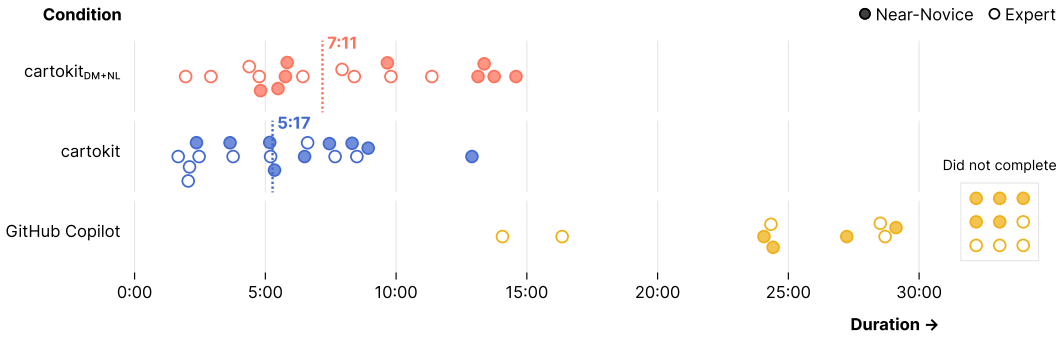


Fig. 2. **Completion times of participants on Reproduction tasks.** Dashed lines and their associated labels denote median completion times across all participants for the condition. GitHub Copilot had a completion rate of 50%; the nine participants that did not complete the task are reflected in the box at right.

5.3 Analysis

Quantitative Analysis. For each Reproduction task, we collected data on completion (True/False) and completion time. Determining completion on a reproduction task like ours can be challenging, as the experimenter's subjective assessment of the faithfulness and accuracy of a participant's map against the target map can be biased. Therefore, we used a rubric with a minimum set of unambiguous criteria that participants should implement to declare the task complete. We did not give participants access to the rubric as we suspected it could aid them in task decomposition, artificially simplifying the task. Instead, participants notified the researcher when they believed they had reproduced the map closely enough. In the event that a participant had not implemented criteria from our rubric, we asked them specifically to implement the functionality. To determine the completion time for the task, we used the timestamp from the video recordings when the participant's JavaScript program successfully executed in StackBlitz and rendered their map.

Qualitative Analysis. In total, we collected 34 hours of session recordings. We then conducted an inductive, reflexive thematic analysis [13] of the video data using the qualitative coding tool MaxQDA [82]. We started with an open coding phase in which the first two authors each independently tagged segments of the session recordings with short, descriptive phrases of participant behaviors. In a second phase, the first two authors met to group their respective open codes into a hierarchy of axial codes. Finally, the entire research met several times to discuss, refine, and regroup these axial codes into top-level themes, discussed in §7.

In addition to the session recordings, we recorded and analyzed all prompts issued by participants in the DM+NL (`cartokit`$_{DM+NL}$) and NL (GitHub Copilot) conditions, as well as telemetry data of direct manipulation and prompt interactions in the DM+NL and DM (`cartokit`) conditions. We present qualitative findings on participant prompts and `diff` sequences throughout §7.

6 Quantitative Results

In this section, we provide quantitative results on task completion rates and times across conditions. We also present self-reported NASA Task Load Index scores for each condition-task pair.

6.1 Task Completion and Timing

RQ1 *Given the task of reaching a fixed target program, how does the* combination *of direct manipulation and natural language as editing modalities affect participants' success rate or completion time compared to direct manipulation or natural language alone?* Participants were broadly more

successful at completing Reproduction tasks with $\texttt{cartokit}_{\text{DM+NL}}$ (18/18, 100%) and `cartokit` (18/18, 100%) than with GitHub Copilot (9/18, 50%). Of the nine participants that did not complete the Reproduction task with GitHub Copilot, six (66.$\overline{6}$%) chose to abandon the task before the 30-minute threshold.

Participants also completed Reproduction tasks faster with $\texttt{cartokit}_{\text{DM+NL}}$ and `cartokit` than with GitHub Copilot. Figure 2 shows the completion times of all participants across Reproduction tasks. The median completion time with $\texttt{cartokit}_{\text{DM+NL}}$ was 7m11s, while participants were somewhat faster with `cartokit` (5m17s). As mentioned above, only nine of 18 participants completed the task with GitHub Copilot. Given that this data is right censored at *exactly* 50% completion, we can only estimate the median completion time. To do this, we conducted a survival analysis with a Kaplan-Meier estimator [41] and used Greenwood's formula [29] to derive a 95% confidence interval on the median. The resulting estimated median completion time exceeds the timing threshold (30m) and is thus reported as $+\infty$ with a 95% CI = [24m25s, $+\infty$].

Effect Size. To compute an effect size between each pair of conditions, we conducted a second survival analysis on the pairwise differences, per participant, in completion times. We then bootstrapped our Kaplan-Meier estimator using the bias corrected and accelerated bootstrap [22] to derive 95% confidence intervals. Figure 3 shows the results. Participants were 4m23s slower (95% CI = [-1m58s, 7m36s]) using $\texttt{cartokit}_{\text{DM+NL}}$ than `cartokit`, 12m9s faster (95% CI = [-22m45s, -11m40s]) using $\texttt{cartokit}_{\text{DM+NL}}$ than GitHub Copilot, and 20m47s faster (95% CI = [-23m40s, -13m35s]) using `cartokit` than GitHub Copilot.

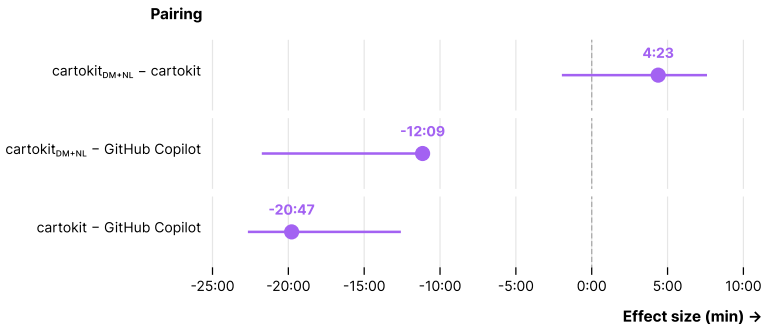


Fig. 3. **Median difference in completion times (effect size) between conditions on Reproduction tasks.** Negative values indicate participants were faster in the LHS condition than the RHS condition (and vice versa). Purple lines show 95% CIs.

6.2 NASA Task Load Index

Participants self-reported perceived workload after each Reproduction and Exploration task, using ratings from 0-20 along the six subscales used in the NASA-TLX: Performance, Effort, Frustration, Mental Demand, Temporal Demand, and Physical Demand. Figure 4 shows the distributions of participant ratings by subscale. Lower values denote less perceived workload or, in the case of the Performance subscale, *better* perceived performance. We then computed the unweighted mean of these values as the composite Raw TLX score.

On Reproduction tasks, participants self-reported a mean Raw TLX score of 4.86 (SE ±0.40) with $\texttt{cartokit}_{\text{DM+NL}}$, 4.33±0.51 with `cartokit`, and 10.02±0.71 with GitHub Copilot. On Exploration tasks, participants self-reported a a mean Raw TLX score of 4.95±0.58 with $\texttt{cartokit}_{\text{DM+NL}}$, 4.88±0.67 with `cartokit`, and 10.51±0.63 with GitHub Copilot. In addition, we compare the differences in mean Raw TLX scores between conditions per task type, reporting 95% confidence intervals using the bias corrected and accelerated bootstrap (Table 2). In general, our results show that participants' perceived workload in the NL condition (GitHub Copilot) was markedly higher

Table 2. **Differences in means of participant Raw TLX scores, paired by condition and task type.** Negative differences indicate that participants' perceived workload on the LHS condition was lower than on the RHS condition (and vice versa).

Pairing	Diff. Mean Raw TLX [95% CI] Reproduction	Diff. Mean Raw TLX [95% CI] Exploration
$\texttt{cartokit}_{\text{DM+NL}}$ — `cartokit`	`0.53 [-0.94, 1.44]`	`0.07 [-0.72, 0.95]`
$\texttt{cartokit}_{\text{DM+NL}}$ — GitHub Copilot	`-5.16 [-6.44, -3.53]`	`-5.56 [-7.17, -4.06]`
`cartokit` — GitHub Copilot	`-5.69 [-7.08, -3.91]`	`-5.63 [-7.28, -4.20]`

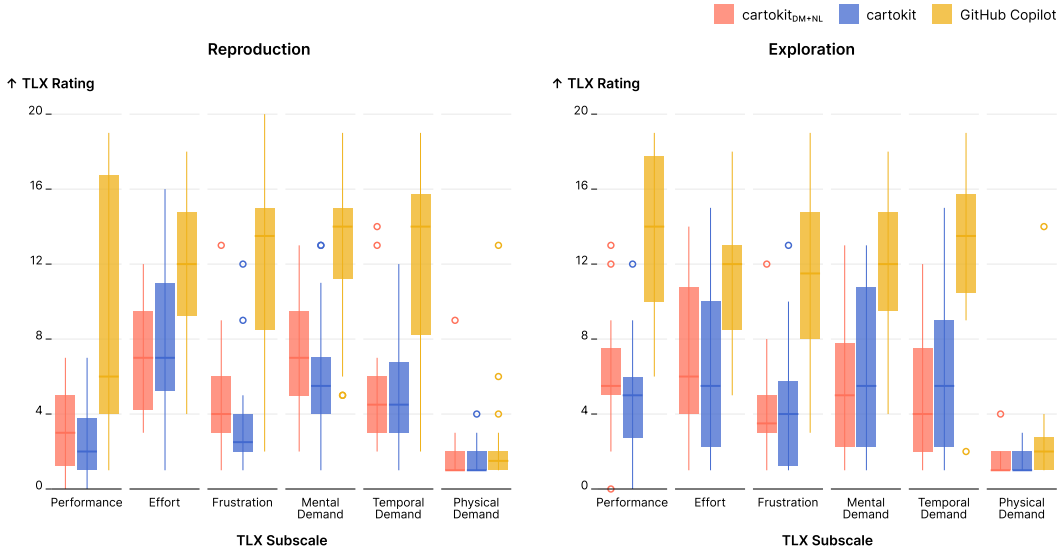


Fig. 4. **Distribution of participant NASA-TLX scores by subscale and condition, on Reproduction tasks (left) and Exploration tasks (right).** Lower values indicate less perceived workload or, in the case of the Performance subscale, better performance.

than both the DM+NL ($\texttt{cartokit}_{\text{DM+NL}}$) and DM (`cartokit`) conditions, regardless of task type. However, there is no statistically meaningful difference between the DM+NL and DM conditions, suggesting that the addition of natural language editing capabilities in $\texttt{cartokit}_{\text{DM+NL}}$ was not meaningfully associated with an increase or reduction in participants' perceived workload.

7 Qualitative Results

In this section, we discuss qualitative findings from our within-subjects study, focusing in particular on how the DM+NL condition shaped participants' programming process relative to the DM and NL conditions (**RQ2**), and how behaviors differed between near-novices and experts (**RQ2a**). Throughout this section, we make references to participants' prompts, denoted by periwinkle text.

7.1 How did participants edit programs?

7.1.1 In the DM+NL condition, participants overwhelmingly used direct manipulation editing, but experimented with natural language editing to automate repetitive or long-range tasks. In total, participants issued 55 prompts across all tasks in the DM+NL condition, amounting to ≈1.5 prompts

per participant per task. Of the 2,946 `diffs` dispatched by participants in this condition, only 181 (≈6.14%) were natural language edits. Qualitative analysis of participant behaviors largely reflected this skew. We observed the majority (14/18) of participants started their editing process with direct manipulation, with only four participants opting to start with prompting; some participants never prompted in the course of a task. When asked about avoiding natural language editing, participants explained that they often already understood how to achieve their desired edit through direct manipulation, so it "didn't feel faster" (PE1)—and in fact, it "felt like I'm adding layers of abstraction to what I'm trying to do" (PE4)—to describe the edit in natural language. Among participants who did prompt, we observed that they primarily leveraged natural language editing to (i) automate repetitive tasks that would be tedious to perform via direct manipulation or (ii) scaffold long-range edit sequences. §7.2 dives deeper into these behaviors, providing a detailed analysis of when, how, and why participants switched between direct manipulation and natural language.

7.1.2 In the DM+NL and DM conditions, the GUI scaffolded how participants edited programs, guiding them from "high-abstraction" to "low-abstraction" edits. Participants were remarkably consistent in their approach to editing programs in both $\texttt{cartokit}_{\text{DM+NL}}$ and `cartokit`, with the vast majority turning first to GUI controls supporting high-abstraction edits, such as modifying layer types and encoding channels, and moving progressively towards controls supporting low-abstraction edits, like adjusting colors, opacities, and symbol size ranges. The former category of edits are "high-abstraction" in the sense that they abstract over many concrete details of the data encoding strategy, whereas the latter correspond to styling "tweaks" of specific visual properties. While the GUI made performing both classes of edits similarly efficient for participants, their impacts on the program were quite different. High-abstraction edits often introduced larger, non-local changes to the program, such as introducing new function definitions and call sites to transform or analyze data, whereas low-abstraction edits had localized effects on individual strings or number literals.

We hypothesize that this high-abstraction to low-abstraction editing pattern reflects how the GUI in both $\texttt{cartokit}_{\text{DM+NL}}$ and `cartokit` both decomposed and scaffolded the programming process for participants. On Reproduction tasks, many participants (14) navigated the layer editing GUI sequentially, treating each GUI control as a distinct editing choice as they moved closer to the target map. On Exploration tasks, participants made heavy use of the top-level Layer Type controls to transition quickly between standard geospatial visualizations, then shifted to iteratively tweaking details like color schemes or symbol sizing as they refined their maps. Importantly, the GUI itself helped to communicate the kinds of edits that were possible, shifting participants' main programming task from recall (of syntax) to recognition (of GUI controls) [14]. In addition, by implicitly guiding participants through a sequence of editing decisions, the GUI absorbed much of the problem decomposition burden that prior work has shown to be a central obstacle in natural language programming, particularly for near-novices [4, 42, 57]. We suspect that this was a key factor in participants' strong performance in both the DM+NL and DM conditions.

***Comparison**: In the NL condition, participant prompts attempted to perform large-scale changes all at once, which could lead to erring programs or regressions in functionality.* Participants regularly issued prompts that tried to make entire maps in "one shot." Even in instances where they effectively decomposed a target map into discrete components (e.g., layers, projections, data encodings), going "too big" (PE8) in any single prompt could result in updated programs that regressed in functionality (PE5), removed desired functionality (PE4, PE7), or introduced run time errors that produced no program output (PNN6, PNN8, PE6, PE8). For example, PE6 was making steady progress towards completing a Reproduction task, with both provided datasets successfully rendering on the map. They then moved to specify the size and color encodings of features in one of the layers ("Scale each square by abundance and color by population change."). This prompt resulted in a JavaScript

program that triggered an out-of-memory exception at run time. Concerned, PE6 issued a follow-up prompt to complete a simpler part of the task ("color the range as grey"), explaining, "I want to get back to a working map after I messed it up, then I'll work on the color scheme." However, this prompt resulted in a program that failed to render a map, instead rendering only a legend. The potential for single prompts to have such dramatic impacts on the state of programs was disorienting and disconcerting to participants. As PE6 reflected in the post-interview:

> "I became very frustrated by the fact that I could make all this good progress, and then I would put in a bad prompt or it would read it wrong, and then all of a sudden I've lost tether of that good progress and I can't really build on it … I felt like I had no control of what was going on."

Even when large-scale edits effected by GitHub Copilot did not lead to erring programs, they could have other negative impacts that made progress difficult. For example, PE5 had successfully completed most of the Reproduction task with GitHub Copilot, developing a proportional symbol map that sized point locations of power plants in the United States according to their `total_capacity` property. To wrap up, they issued a prompt to apply a categorical color scheme to the layer. While the resulting generated code correctly applied this change, it simultaneously *removed* the proportional sizing of symbols (PE5: "Well, we took one step forward and one step back here Copilot.") This potential to make progress on some fronts but regress on others again contributed to participants' sense of precarity when prompting; for experts, it was also a point where they abandoned GitHub Copilot in favor of direct textual editing (PE1, PE2, PE5, PE7).

7.1.3 Incremental editing via direct manipulation in DM+NL and DM offered continuity that seemed to affect participant understanding of programs. Direct manipulation interfaces, by definition, support editing via "rapid, reversible, incremental actions" [76] that allow users to "see immediately if their actions are furthering their goals, and if not … [to] change the direction of their activity" [39]. In the context of programming, we observed that this quality of direct manipulation also seemed to help participants reason about how edits they made to the program *output* corresponded to transformations on the program itself. For example, while making changes to the attribute (`votes_total` → `pct_dem_lead`) and classification-method (Manual → Equal Interval) parameters of a choropleth color scale in the GUI, PNN4 carefully observed how this edit altered the numeric values in the associated `fill-color` expression in the generated JavaScript code. Many participants employed a similar strategy of observing program updates when adding a new data layer to the map (PNN1, PNN2, PNN8), switching map projections (PNN6), or transitioning layer types (PNN1) in the GUI. As PE6 explained, the strategy was a useful tactic for bootstrapping program understanding:

> "I really like the fact that I can see the code live updating as I'm putting in new values. That feedback is so good for someone like me who eventually would like to know how to code using MapLibre. Having that instant feedback is really, really helpful as a learner."

Importantly, the *incremental* nature of direct manipulation edits appeared to be critical in this process. Because direct manipulation edits corresponded to conceptually discrete changes on the output that participants initiated one at a time, many could successfully identify the set of locations in the generated JavaScript program that had changed in response to their interaction.

***Comparison**: In the NL condition, participants attempted to "incrementalize" natural language edits.* Interestingly, several participants tried to emulate the incremental nature of editing in direct manipulation in their interactions with code generated by GitHub Copilot. The most common behavior involved a form of manual program slicing, where participants copied only relevant "slices" of the generated code (e.g., a function definition and its call sites) from GitHub Copilot to their working environment in StackBlitz (PNN4, PNN7, PE1, PE7, PE8). Because this strategy required

being able to (i) identify relevant slices in a large program based on a specific sub-goal and (ii) trace slices to extract required dependencies, it came with risks; both PNN4 and PE1 introduced errors through manual slicing that were not present in the generated code. But participants also noted that this strategy came with the benefit of forcing incremental verification of small parts of the program, which aided in error localization. For example, PNN4 constructed their program by first copying over just the slice of the generated code to instantiate the `maplibregl.Map` class. When this first slice did not produce a map onscreen, they noticed that the GitHub Copilot-generated code assumed that `maplibregl` was already available in the top-level program scope or on the `window Object`. They fixed this error by adding an `import` statement to their program to bring the `maplibregl` module into scope; they then continued "slicing" by bringing over relevant code for adding a data source in a `map.on('load')` callback and verifying the callback executed.

Experts explained that this slicing strategy was an attempt to recover patterns of how they already write code using external resources like StackOverflow or examples from online documentation. In contrast, the near full program rewrites of iterative prompting with GitHub Copilot broke this cycle, encouraging full program copy-pasting. PE8:

> "It's [GitHub Copilot] throwing a lot at me, and I think that when I'm coding I like to kind of go line-by-line and like—if I'm pulling code over from another source I usually will pull things over in small little chunks to make sure I know which part is breaking it. Like, kind of break it as I go and fix it as I go. So it's challenging to pull over an entire script and then try to diagnose the problem."

Some participants also tried to "incrementalize" outputs from GitHub Copilot by constraining the LLM at the prompt level. For example, PE7 explicitly requested only a snippet of code as a response from GitHub Copilot in lieu of full program regeneration: "Give me JUST the color scale stuff".

7.2 What caused participants to switch between direct manipulation and natural language when both were available?

The simultaneous availability of direct manipulation and natural language as program editing paradigms in $\texttt{cartokit}_{\text{DM+NL}}$ allowed us to examine when, how, and why programmers switch between the two. Based on both direct observation and telemetry data (Figure 5) of direct manipulation and natural language edits, we articulate several patterns of use.

DM Only: Some participants never switched to NL. Several participants elected to use direct manipulation only and did not issue a single prompt to the LLM in the course of completing the Reproduction (PNN2, PE3 PE7, PE8) or Exploration task (PNN1, PNN5, PNN7, PE6, PE7). The most common explanation for avoiding the LLM from participants was that they already knew how to achieve their particular aim with direct manipulation, so turning to the model felt like an unnecessary detour. PE7 explained:

> "I definitely did forget that [the LLM] was a feature, but also, looking back, I don't know what I really would've needed it for because I was directly manipulating the map and I didn't need help to figure that out … I thought I could just accomplish everything without having to declare bankruptcy and be like, 'You [LLM] tell me how to do this, I don't really know.'"

DM→NL: Some participants used natural language as a last resort. A small handful of participants struggled to discern how to perform a particular program edit via direct manipulation in $\texttt{cartokit}_{\text{DM+NL}}$ and, as a last resort, turned to the LLM for help (PNN4, PNN7, PNN8). For example, PNN8 wanted to map the `fill` channel of their active layer to a categorical color encoding using the (categorical) `primary_source` property on their dataset. However, at the time of edit, the layer was set to use a quantitative color encoding; in this state, the $\texttt{cartokit}_{\text{DM+NL}}$ GUI preemptively filters out non-quantitative variables. Puzzled that `primary_source` did not appear as an option in

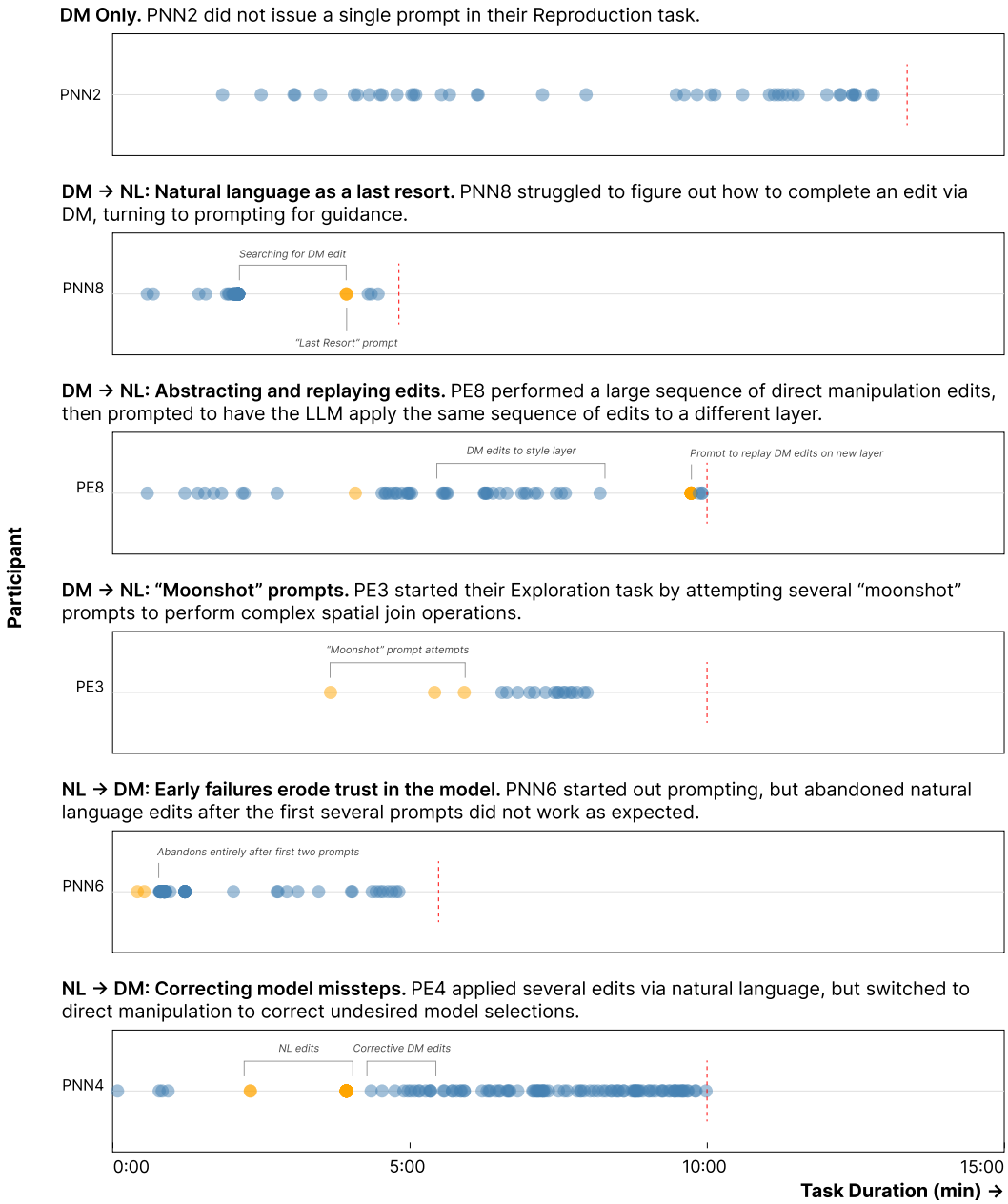


Fig. 5. **Examples of participants switching between direct manipulation ● and natural language ● edits in cartokit$_{DM+NL}$.** Dashed red lines indicate when the participant successfully completed the task.

the GUI, PNN8 turned to cartokit$_{DM+NL}$'s LLM: "How can I show the power source of each power plant on the map?" GPT-5 correctly sequenced three `diffs` in response to this prompt to modify the program and output to use the categorical color encoding. In response, PNN8 exclaimed: "Aha! Wow! It just did it for me, it didn't tell me how … Cool, yeah, I couldn't figure that out."

Still, many participants had quite high tolerance for struggling with direct manipulation in $\texttt{cartokit}_{\text{DM+NL}}$, opting to stick with it even when interface navigation was a core challenge. For example, PNN2, who took the second longest to complete the Reproduction task using $\texttt{cartokit}_{\text{DM+NL}}$ and had the most difficulty making direct manipulation edits, did not issue a single prompt in the course of the task. When asked why they did not turn to the LLM for assistance, they explained: "I wasn't sure how to prompt it. I wasn't sure if it [the prompt] had to be in technical terms or if you could just say whatever and it still worked."

DM→NL: Participants used NL to automate or replay repetitive tasks. After participants developed an understanding of how to complete larger-scale program edits via repetitive direct manipulation interactions, some turned to natural language to automate performing similar edits (PNN4, PNN5, PNN6, PE4, PE5) or replaying edits on new data (PNN3, PNN4, PE4, PE6, PE8). An example of the former came up when participants had to set specific thresholds for quantitative breaks in the color scale of a choropleth map, requiring six to nine direct manipulation interactions with multiple `<input>` elements. This kind of tedious, repetitive work was emblematic of tasks participants wanted to offload to the LLM. Natural language also allowed participants to leverage higher-level descriptions of the repetitive task rather than specifying precise literal values. For example, PE5 directed the LLM to set breaks at "integer values in 10 percentage-point increments moving away from zero" rather than specifying each threshold individually.

The latter use case—replaying edits on new data—involved participants trying to use the LLM almost like a parameterized record-and-replay system [6, 17, 18]. In these instances, participants often spent significant time adjusting the symbology of one layer via direct manipulation, and then prompted the LLM to copy the symbology to a second layer while accounting for differences in the underlying datasets. The goal, again, was to avoid repeating many direct manipulation edits and instead delegate them to the model. PNN4 explained:

> "Because the clickable interface is so easy to navigate, I only really felt compelled to [issue a prompt] when there was something tedious in front of me … Like, I wanted to make the 2020 and 2024 [election data layers'] color bars the same. Like, ugh, that's gonna be a pain. That's gonna be a perfect, like, ask an LLM to change it."

Prior work has speculated that this delegatory pattern of use could be a primary benefit to systems blending direct manipulation and natural language [25]. While our empirical evidence confirms this, participants also expressed an extremely low tolerance for model errors in this context, explaining that almost any amount of error would obviate the automation benefit (PNN1, PNN4, PE4, PE8).

DM→NL: Participants used NL for "moonshot" prompts. Particularly on unbounded Exploration tasks, several participants turned to $\texttt{cartokit}_{\text{DM+NL}}$'s LLM to attempt "moonshot" prompts—computation that they presumed was not possible through direct manipulation but thought could be plausibly handled by an LLM. These tended to be more complex geospatial analysis operations that can be orchestrated (with significant expertise!) in full-fledged desktop GISs, spatial databases, or Python libraries, including spatial or tabular joins across layers (PE3), computation of aggregate values or indices across layers (PNN8, PE3, PE4, PE5), or bivariate symbologies (PE5). Some participants indicated that they did not necessarily expect these "moonshot" prompts to work, but used them as a proactive way to "temperature check" (PE4) or calibrate the LLM's capabilities.

NL→DM: Early failures eroded participants' trust in the NL tooling. Many participants abandoned using $\texttt{cartokit}_{\text{DM+NL}}$'s LLM entirely after encountering failures (PNN1, PNN4, PNN6, PNN7, PE1, PE2, PE3, PE4, PE5, PE6), whether that came in the form of the reserved **unknown** `diff` (signaling that GPT-5 could not construct a `diff` sequence based on the prompt) or a timeout on response from the model. Some participants' tolerance for failure was extremely limited—even one failure

was enough to incentivize abandoning. As PNN6 explained, success or failure of the first prompt had an outsized impact on participants' perception of model capability and utility: "The first time you try to use it is such an important evaluation point, y'know? 'Cuz I was like, 'I'm gonna try this' and then it's like, 'Error.' and I was like, 'Ok, not using that.'" This is consistent with findings in prior work that just one or two failures are sufficient for users to presume model incapability [90]. While other participants were more persistent in model use and engaged in systematic testing, we still observed consistent drop-offs in prompting over the course of tasks. Our hypothesis is that, in many instances, direct manipulation provided a comparatively easy pathway to progress—often even easier than direct text editing—when the model did fail, encouraging even quicker abandonment.

NL→DM: Participants used DM to correct or repair model missteps. Due to the inherently ambiguous nature of natural language, it was common for the $\texttt{cartokit}_{\text{DM+NL}}$ LLM to partially succeed—that is, it applied some of the `diffs` the participant expected, but either failed to apply others or applied undesired `diffs`. In these instances, participants turned quickly to direct manipulation to correct the model's missteps (PNN1, PNN4, PNN8, PE5, PE8). For example, PE8 attempted to transfer the symbology of one layer to another using natural language, similar to PNN4 above ("Make the 2020 swing states layer match the symbology of the 2024 swing states, adjusting for the fact that the pct_dem_lead is scaled differently."). While the LLM managed to match the symbology of the two layers quite closely, PE8 could tell by looking at the updated map that certain discrepancies remained; in this instance, $\texttt{cartokit}_{\text{DM+NL}}$'s LLM had failed to apply three `diffs` to adjust the `fill-opacity`, `method`, and `scheme-direction` of the target layer. PE8 turned immediately to direct manipulation to apply these lingering edits that the LLM had missed.

7.3 Experience with direct manipulation editing helped mitigate known challenges with natural language editing

Two of the most well-studied obstacles identified in the literature on human-LLM interactions in natural language programming contexts involve (i) inferring model capabilities [46, 57, 90] and (ii) interpreting model outputs [58, 94]. In this section, we focus on how participants' experiences with direct manipulation helped mitigate these challenges in both the DM+NL and NL conditions.

7.3.1 Prior exposure to the DM+NL and DM conditions improved success of prompting in the NL condition. We observed that participants with prior exposure to either $\texttt{cartokit}_{\text{DM+NL}}$ or `cartokit` (in earlier conditions within the study session) often decomposed their natural language prompts in GitHub Copilot along `cartokit`'s abstraction boundaries, referencing concepts like layers, layer types, encoding channels, and classification methods (PNN2, PNN3, PNN4, PNN5, PNN6, PNN9, PE2, PE4, PE5, PE6). For example, PE2, who interacted first with $\texttt{cartokit}_{\text{DM+NL}}$, issued the following first prompt to GitHub Copilot:

> "Make a choropleth map using the following US unemployment data. Use the rate property to determine the darkness of the polygon color. Use red. Darker colors should correspond with higher numbers. Bin the colors along quintiles. <Data URL>"

The structure of this prompt maps almost directly to the order in which PE2 interacted with direct manipulation controls in the $\texttt{cartokit}_{\text{DM+NL}}$ interface in the prior task, which they used to specify a layer type ("choropleth map"), a GeoJSON property to map to the `fill` channel ("Use the rate property to determine the darkness of the polygon color."), a color scheme and direction for the `fill` channel ("Use red. Darker colors should correspond with higher numbers."), and a classification method for binning continuous numeric values into discrete bins ("Bin the colors along quintiles.").

Our hypothesis based on this behavior is that `cartokit`'s direct manipulation GUI provided participants with effective scaffolding on *how* to decompose a target map at a particular abstraction

level, which they learned quickly enough to apply in a natural language setting. PNN4 confirmed this, describing $\texttt{cartokit}_{\text{DM+NL}}$ as a "killer teaching tool" that helps direct users when they have only "half a picture of what they're trying to get out of their map." Beyond map decomposition, `cartokit`'s GUI also gave participants access to technical cartographic and geostatistical terminology that they later used in prompts, particularly to describe the class of visualization (e.g., choropleth, dot density, proportional symbol) they sought from the LLM. Providing both a decomposition and technical language appeared to be especially valuable for near-novices; of the four near-novices that completed the Reproduction task with GitHub Copilot, three did so after seeing both `cartokit` and $\texttt{cartokit}_{\text{DM+NL}}$. The fourth, despite being a near-novice in our study, is a professional cartographer that presumably already has access to both map decomposition skills and technical terminology.

7.3.2 Participants used the GUI in the DM+NL condition to make guesses about "allowable" prompts, improving success. Participants expressed uncertainty when prompting the LLM in $\texttt{cartokit}_{\text{DM+NL}}$, citing confusions about "what the model can see" and "[*hovering over program, map, and GUI components*] what interface [is] the recipient of my prompts" (PE5). Lacking knowledge of system internals, several participants looked to direct manipulation features in the $\texttt{cartokit}_{\text{DM+NL}}$ GUI to infer the kinds of program edits they could ask the LLM to make (PNN1, PNN3, PE1, PE4, PE5). Typically, this took the form of directing the LLM to perform edits that mapped very directly to specific GUI components. For example, one of PNN3's prompts—"can you size the electricity sources points with graduated symbols, using the 'total_capacity' attr as the size"—maps directly to clicking the `electricity_sources__1` layer on the map, adjusting its Layer Type via a `<select>` element, and adjusting the attribute on its size channel also via a `<select>` element. When asked about how they reasoned about prompting the LLM in $\texttt{cartokit}_{\text{DM+NL}}$, PNN3 explained:

> "Knowing what the tool was capable of before I started prompting was important. I wasn't doing joins and different spatial analysis things because I didn't see those tools in [the GUI]. I was constrained by what I knew I could do without the chatbot."

In several cases, we observed participants keeping the layer editing controls visible *while* writing their prompts, possibly as a reference aid (PE8, PNN4). We hypothesize that this property of direct manipulation interfaces—that they "announce" what is possible through the interface itself, thereby providing a "language" of interaction—can help bridge the ambiguity gap between users and models.

7.3.3 Participants used the GUI in the DM+NL condition to interpret model-triggered edits. When participants did turn to the LLM to perform certain program edits, $\texttt{cartokit}_{\text{DM+NL}}$'s GUI was essential in helping them determine what the model did (PNN3, PNN4, PNN8, PE5, PE8). For example, after issuing the prompt "Can you bin the data in '2024 election results' by increments of 0.25?" and seeing their map update, PNN3 turned to the GUI to understand what edits the LLM had applied. Nearly immediately, they could tell the edits were only partially successful: "Oh! Ah … well, ok, something changed … Oh, ok, so it switched it [the classification method] to Equal Interval. Oh, cuz maybe, I did say the word 'increment.'" Understanding model edits through the GUI was effective for participants for two reasons. First, changes in the GUI were often easier for participants to quickly check for correctness compared to examining the generated JavaScript program. Interestingly, while participants also examined changes to the map to understand model edits (PE2, PNN1, PE5, PE8), they often were not able to understand the scope of edits from the map alone. Second, as the above anecdote illustrates, participants used their understanding of model-trigged edits from the GUI to develop hypotheses about how their natural language prompts were being interpreted by the system. PE5 described this process, sometimes called "abstraction matching" in the literature [46, 72], as "learn[ing] their [the LLM's] rules through some process of discovery," and noted that the GUI was particularly useful for this process.

7.4 How did participants assess correctness?

Researchers and industry practitioners have targeted programming domains producing visual outputs—such as UI design [66, 67], web development [83], and data visualization [74]—as good fits for natural language programming, presuming that outputs alone are sufficient for non-experts to assess program correctness. While we did observe that output examination was the primary way participants proxied for correctness in all conditions, we also found that this tactic was *not* always sufficient, a finding in line with a recent study of how domain experts program with LLMs [58]. In this section, we discuss challenges participants hit gauging program correctness.

7.4.1 Soundness guarantees in the the DM+NL and DM conditions allowed participants to focus on the output. The $\texttt{cartokit}_{\text{DM+NL}}$ and `cartokit diffs` are designed so that programmers cannot reach invalid programs in these systems and, consequently, cannot reach a state where the program output (i.e., the map) is not visible. For this reason, participants never entered an obvious debugging phase using these systems, nor did they express concerns about the correctness of generated code. Instead, "debugging" often manifested as choosing one program among a (theoretically infinite) space of valid programs constrained by the set of available `diffs`. For this purpose, participants could (and primarily did) rely solely on examining the program output and the GUI state.

Occasionally, changes to the program output triggered by a direct manipulation interaction contradicted participant expectations, triggering closer examination of the program (PNN1, PE3, PE4). For example, after applying a proportional symbol transform to a dataset of US counties with information on broadband access rates, PNN1 noticed that the resulting symbols all appeared to be the same size despite expecting them to be variable. Confused, they turned their attention to the program. Through careful examination, PNN1 noticed that $\texttt{cartokit}_{\text{DM+NL}}$ was using a linear scale to size symbols, with endpoints at the minimum and maximum values of the `broadband_access` variable. Using this information, they inferred that the distribution of `broadband_access` was likely concentrated at the midpoint of the scale, such that many symbols would appear equally-sized. Participants relayed that the relatively small size of generated programs in the DM+NL and DM conditions aided in achieving this degree of code comprehension (PNN1, PE4, PE6).

***Comparison**: In the NL condition, the model could (and did) generate programs that produced no output, leaving near-novices stranded.* Over half of participants reached some point where code generated by GitHub Copilot produced no output at all, commonly due to run time errors such as referencing a non-existent library API, hallucinating a column name on the dataset, or incorrectly transforming dataset geometry. In these cases, participants knew their programs were incorrect but, absent a concrete output to refer to in subsequent prompts, had little way of communicating what was wrong to the model. This was a particularly large obstacle for near-novices, who typically resorted to notifying GitHub Copilot that its code did not produce visible output (e.g., PNN9: "The code doesn't work for me, it didn't show the preview appropriately."), followed up by restating the previous prompt in slightly different language (PNN2, PNN5, PNN9, PE8) or asking the model to self-debug (PNN2, PNN6). These follow ups still tended to fail, as participants could not identify underlying errors (such as non-existent library API usage) that the model was unlikely to correct in the next round of generation. Even when GitHub Copilot included a natural language text summary of edits it made as well as step-by-step debugging instructions for the participant to take, we observed that participants largely ignored this information.

***Comparison**: In the NL condition, the volume of generated code inhibited participants' ability to assess correctness.* Over half of participants also complained about the volume of generated code returned on every prompt in the NL condition, explaining that it made discerning whether the model had implemented the desired functionality more difficult. Often, code verbosity was related

to functionality the participant had not explicitly requested, such as legends, tooltips, unnecessary dataset parsing or transformation, or reimplementations of common geospatial operations (e.g., bounding box computation). Experts noted that this "fluff … made it harder to debug" (PE2) and that GitHub Copilot "seemed to way overcomplicate things" (PE1) For near-novices, the volume of generated code was both intimidating and increased skepticism of model success. In response to receiving a 403-LOC HTML file after prompting GitHub Copilot, PNN1 said, "I'm scared." While watching >600 LOC stream in from GitHub Copilot, PNN2 lamented: "Not convinced by this."

Beyond adding the overhead of unrequested functionality, large-scale code changes made pinpointing true errors more difficult. Because participants had not written the program themselves, they lacked an understanding of the program structure that would support error localization. PE4:

> "Some of [the difficulty of working with LLMs] is that it just vomits a whole ton of code on you at once. So it's like you're looking at an unfamiliar codebase … Like, it was more mentally taxing to try and figure out why Copilot couldn't do what I wanted it to do than it would have been to write the code myself, honestly."

This finding is consistent with observations of experts in [5, 81]. For near-novices, we found that encountering any error largely halted progress because participants (i) struggled to identify and meaningfully articulate the error to the model and, thus, (ii) were dependent on the model to successfully self-debug. Prompts like "It's not displaying. Please fix that" (PNN6) or "the power plants are not showing up on the map" (PNN2) became common at this point.

7.4.2 Participants had concerns about the scope and completeness of model-triggered edits in the DM+NL condition. We observed that the addition of natural language editing in the DM+NL condition—by virtue of producing non-deterministic `diff` sequences—added to the challenge of assessing the scope and completeness of model-triggered edits relative to expectations based on the prompt (PNN1, PNN4, PE4, PE5, PE6). As PE5 remarked, issuing a natural language edit came with the burden "of hav[ing] to go back and confirm that [the LLM] did what you expected it to do, even if you have the visual indicator that maybe it got somewhat close." PE8 described this overhead as introducing a "cost-benefit analysis" to natural language editing, where they had to choose between the benefit of automating larger-scale edits and the cost of having to verify those edits. We also observed that, despite the multiple "lenses" participants had on natural language edits (in the form of changes to the program output, the program, and the GUI state), it was still easy to miss subtle incongruities between their prompts and updated outputs. For example, PE4 issued a prompt to "Style the 2024 layer as a choropleth by the `pct_dem_lead` fields [sic]" but failed to notice that GPT-5 had selected the `votes_total` property for visualization instead. In this instance, because the resulting output map looked plausibly correct, PE4 failed to notice this choice. Conversely, participants in the DM condition never expressed concerns about the scope or completeness of edits because all edits were explicitly programmer-triggered.

8 Limitations

A key challenge in conducting any study involving human-LLM programming interactions is the pace at which model capabilities are evolving. We began conducting study sessions in August 2025, at which time GitHub Copilot with GPT-5 represented a state-of-the-art frontier model for programming; at the time of writing, several new frontier models have since been released. Expecting this, we have focused our analysis on observations related to the programming paradigms themselves (DM+NL, DM, NL) rather than on the specific capabilities of any particular model or tool. For example, nearly all participants remarked on slow responses from GPT-5 when using GitHub Copilot, but, given that this is a limitation of a particular model rather than a limitation of natural language programming more generally, we do not discuss it in our analysis. In this

way, we expect our findings to remain relevant for researchers working at the boundaries of these programming paradigms (and their combinations).

9 Related Work

9.1 User Studies of DM+NL Programming

To our knowledge, there are no user studies exploring programming systems that support interchangeable editing through both direct manipulation and natural language.

9.2 DM+NL Programming Systems

To our knowledge, there are no existing programming systems that support interchangeable editing through both direct manipulation and natural language.

9.3 User Studies of Direct Manipulation Programming

Direct manipulation programming is a rich area of recent research at the nexus of programming languages and human-computer interaction, with researchers developing direct manipulation programming systems for SVG [19, 35, 36, 93], HTML [49, 92], game development [26], and geospatial visualization [96]. However, despite the proliferation of system development in this space, we are not aware of *any user study of direct manipulation programming*. Thus, this work represents an important first step in filling this gap in the literature. Interestingly, our results offer the first evidence for some of the speculated benefits posited in the motivation of prior work (e.g., that direct manipulation programming can in fact scaffold the programming process for near-novice programmers and support some degree of program comprehension) [36].

9.4 User Studies of Natural Language Programming

Research on natural language programming dates back to at least the 1960s [7], and the recent arrival of code-generating LLMs has driven widespread interest in understanding how these technologies shape the programming process. Studies in this space are numerous, with researchers conducting in-lab experiments, observational studies, randomized control trials in the field, evaluations of field deployments, and more, with expert programmers [5, 8, 24, 55, 81, 95], novice programmers [20, 42, 44, 47, 57], domain experts [58, 84], or some combination [88, 89]. Our work builds on these studies by examining how users' natural language programming practices change when we add direct manipulation as an editing modality. Most excitingly, our findings indicate that direct manipulation can alleviate key challenges with natural language programming, including issues with understanding model capabilities [57, 90] and interpreting a model's program edits [58].

9.5 DM+NL Tools Outside of Programming Contexts

Prior work has blended direct manipulation and natural language in interfaces for visualization editing [80, 85], styling web pages [43], vector graphics [48], and animation [12]. For example, DynaVis [80] leverages users' natural language prompts to generate "dynamic widgets," new pieces of UI to support direct manipulation of visualization properties. Their approach uses an LLM for code completion on strict, pre-defined HTML and JavaScript templates, followed by a separate, lightweight program analysis. DirectGPT [48] uses a combination of engineered prompts alongside instrumentation that adds contextual information to text or SVG. Our work differs from these prior systems in two ways. First and most importantly, these tools are not *programming systems* and do not produce programs as primary outputs for users. Second, the approaches these systems employ rely on an LLM to generate *unstructured* output (e.g., completions for function bodies), which provides no guarantees on correctness or compatibility with the host system. In contrast,

our approach uses an LLM to generate sequences of *structured* program edits. We can design the allowable edits to guarantee correctness, obviating the need for additional program analysis.

9.6 Constrained Decoding in Programming Contexts

There has been substantial research exploring how constrained decoding can improve the syntactic and semantic correctness of code generated by LLMs [2, 10, 54, 56, 68] and even support higher-level program authoring through natural language [9]. In general, these tools do not use constrained decoding specifically for generating program *edits* but rather focus on full program synthesis with constraints. Notably, however, these approaches are complementary with our work because `diffs` are an edit *language*. For example, a system developer could use a framework like ChopChop [56] as the constrained decoder to generate `diffs` that with extensive semantic constraints; similarly, in latency-sensitive settings like ours, a constrained decoder like DOMINO [10] could speed up `diff` generation. Thus, we believe that advances in constrained decoding for code generation have substantial implications for improving LLM-backed systems built on `patch-recon`.

9.7 LLMs for Program Editing

Prior work has explored using LLMs for other forms of program editing, including automated program repair [70, 87, 91], automated refactoring [69], large-scale code migration [97], structure editing and live programming [11], and edit prediction [31]. In addition, many industrial tools (e.g., Claude Code [3], GitHub Copilot [28], Codex [62]) focus on program editing in addition to drafting. In general, these systems still produce unstructured token sequences in the host programming language as outputs, as opposed to our approach of generating structured edits in an edit language.

10 Discussion and Conclusion

Is natural language really natural? Our results suggest future work should more deeply explore some of the motivations of prior work on natural language interfaces [45, 52, 74, 85], which claim natural language requires "less prior knowledge" [85], "should have a low learning barrier for end users" [45], and "drastically increas[es] the accessibility of programming to non-expert users" [52]. While these claims sound reasonable, our quantitative and qualitative results suggest a much murkier picture. Although natural language programming may remove the obstacle of learning the syntax and semantics of a programming language, it may also add an entirely new obstacle of learning the informal language of the model (or, as PE5 called it, "learning to play by [the model's] rules"). Assessing the "naturalness" of natural language is far from the central research question of this work, but our findings do reinforce the growing body of literature calling its naturalness into question [23, 46, 47, 57]. Since our work juxtaposed natural language programming with direct manipulation programming and offered preliminary evidence that participants did not face the same discoverability challenges with the latter, future work may want to use direct manipulation as a point of comparison, even a baseline.

Direct manipulation and natural language programming, together at last? Our results offer a first glimpse of the benefits of combining direct manipulation and natural language programming, but they also demonstrate that direct manipulation can be a powerful programming paradigm in its own right. This is perhaps not surprising given the broad adoption of direct manipulation interfaces in everyday computer use, and yet we did not find empirical evidence for this finding in the literature. Foundational work on direct manipulation programming [19, 35, 36, 49, 92, 93, 96] developed the technical foundations of the paradigm but primarily evaluated expressiveness; building on those foundations, our work goes a step further by examining how the paradigm supports a diverse set of programmers *in practice*. Encouragingly, our findings hint that direct

manipulation programming may be able to live up to its promise of simplifying programming relative to alternative paradigms; in some contexts, programmers may even prefer direct manipulation over natural language programming.

Still, we aim to move away from framings that position direct manipulation and natural language as oppositional approaches (e.g., [77]). Rather, we see a future where both coexist as complementary program editing paradigms in increasingly sophisticated programming systems. This paper offers a first look at what writing code with such systems entails for programmers, and provides a possible blueprint for designing these systems based on the idea of a shared language of structured edits.

11 Data-Availability Statement

The source code for the system described in §4 will be made publicly available and submitted for Artifact Evaluation. We will also publish a separate repository on Zenodo that will include participant timing data and NASA-TLX scores, implementations of our quantitative analyses and statistical models, scripts for reproducing our figures, a static snapshot of the database containing participant prompts and `diffs`, and user study materials (e.g., tutorials and task descriptions).

Acknowledgments

References

[1] Michael D. Adams, Eric Griffis, Thomas J. Porter, Sundara Vishnu Satish, Eric Zhao, and Cyrus Omar. 2025. Grove: A Bidirectionally Typed Collaborative Structure Editor Calculus. *Proc. ACM Program. Lang.* 9, POPL, Article 73 (Jan. 2025), 29 pages. doi:10.1145/3704909

[2] Lakshya A Agrawal, Aditya Kanade, Navin Goyal, Shuvendu K. Lahiri, and Sriram K. Rajamani. 2023. Monitor-Guided Decoding of Code LMs with Static Analysis of Repository Context. In *Proceedings of the 37th International Conference on Neural Information Processing Systems (NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, 32270–32298.

[3] Anthropic. 2026. Claude Code. https://claude.com/product/claude-code. Accessed: 2026-03-16.

[4] Hannah McLean Babe, Sydney Nguyen, Yangtian Zi, Arjun Guha, Molly Q Feldman, and Carolyn Jane Anderson. 2024. StudentEval: A Benchmark of Student-Written Prompts for Large Language Models of Code. In *Findings of the Association for Computational Linguistics: ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 8452–8474. doi:10.18653/v1/2024.findings-acl.501

[5] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proc. ACM Program. Lang.* 7, OOPSLA1, Article 78 (April 2023), 27 pages. doi:10.1145/3586030

[6] Shaon Barman, Sarah Chasins, Rastislav Bodik, and Sumit Gulwani. 2016. Ringer: Web Automation by Demonstration. *ACM SIGPLAN Notices* 51, 10 (Oct. 2016), 748–764. doi:10.1145/3022671.2984020

[7] M. P. Barnett and W. M. Ruhsam. 1968. A Natural Language Programming System for Text Processing. *IEEE Transactions on Engineering Writing and Speech* 11, 2 (Aug. 1968), 45–52. doi:10.1109/TEWS.1968.4322334

[8] Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein. 2025. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. doi:10.48550/arXiv.2507.09089 arXiv:2507.09089 [cs].

[9] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2023. Prompting Is Programming: A Query Language for Large Language Models. *Proc. ACM Program. Lang.* 7, PLDI, Article 186 (June 2023), 24 pages. doi:10.1145/3591300

[10] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2024. Guiding LLMs the Right Way: Fast, Non-Invasive Constrained Generation. In *Proceedings of the 41st International Conference on Machine Learning (ICML'24, Vol. 235)*. JMLR.org, Vienna, Austria, 3658–3673.

[11] Andrew Blinn, Xiang Li, June Hyung Kim, and Cyrus Omar. 2024. Statically Contextualizing Large Language Models with Typed Holes. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 288 (Oct. 2024), 31 pages. doi:10.1145/3689728

[12] Samuelle Bourgault, Li-Yi Wei, Jennifer Jacobs, and Rubaiat Habib Kazi. 2025. Narrative Motion Blocks: Combining Direct Manipulation and Natural Language Interactions for Animation Creation. In *Proceedings of the 2025 ACM Designing Interactive Systems Conference (DIS '25)*. Association for Computing Machinery, New York, NY, USA, 1366–1386. doi:10.1145/3715336.3735766

[13] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative Research in Psychology* 3, 2 (Jan. 2006), 77–101. doi:10.1191/1478088706qp063oa Publisher: Routledge _eprint: https://doi.org/10.1191/1478088706qp063oa.

[14] Raluca Budiu. 2026. Memory Recognition and Recall in User Interfaces. https://www.nngroup.com/articles/recognition-and-recall/. Accessed: 2026-02-25.

[15] Ernesto A. Bustamante and Randall D. Spain. 2008. Measurement Invariance of the Nasa TLX. *Proceedings of the Human Factors and Ergonomics Society Annual Meeting* 52, 19 (Sept. 2008), 1522–1526. doi:10.1177/154193120805201946

[16] H. Butler, M. Daly, A. Doyle, Sean Gillies, T. Schaub, and Stefan Hagen. 2016. *The GeoJSON Format.* Request for Comments RFC 7946. Internet Engineering Task Force. doi:10.17487/RFC7946 Num Pages: 28.

[17] Sarah Chasins, Shaon Barman, Rastislav Bodik, and Sumit Gulwani. 2015. Browser Record and Replay as a Building Block for End-User Web Automation Tools. In *Proceedings of the 24th International Conference on World Wide Web (WWW '15 Companion)*. Association for Computing Machinery, New York, NY, USA, 179–182. doi:10.1145/2740908.2742849

[18] Sarah E. Chasins, Maria Mueller, and Rastislav Bodik. 2018. Rousillon: Scraping Distributed Hierarchical Web Data. In *Proceedings of the 31st Annual ACM Symposium on User Interface Software and Technology (UIST '18)*. Association for Computing Machinery, New York, NY, USA, 963–975. doi:10.1145/3242587.3242661

[19] Ravi Chugh, Brian Hempel, Mitchell Spradlin, and Jacob Albers. 2016. Programmatic and Direct Manipulation, Together at Last. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '16)*. Association for Computing Machinery, New York, NY, USA, 341–354. doi:10.1145/2908080.2908103

[20] Paul Denny, David H. Smith, Max Fowler, James Prather, Brett A. Becker, and Juho Leinonen. 2024. Explaining Code with a Purpose: An Integrated Approach for Developing Code Comprehension and Prompting Skills. In *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1 (ITiCSE 2024)*. Association for Computing Machinery, New York, NY, USA, 283–289. doi:10.1145/3649217.3653587

[21] Shihan Dou, Haoxiang Jia, Shenxi Wu, Huiyuan Zheng, Muling Wu, Yunbo Tao, Ming Zhang, Mingxu Chai, Jessica Fan, Zhiheng Xi, Rui Zheng, Yueming Wu, Ming Wen, Tao Gui, Qi Zhang, Xipeng Qiu, and Xuanjing Huang. 2026. What is wrong with your code generated by large language models? An extensive study. *Science China Information Sciences* 69, 1 (Jan. 2026), 112107. doi:10.1007/s11432-025-4632-8

[22] Bradley Efron. 1987. Better Bootstrap Confidence Intervals. *J. Amer. Statist. Assoc.* 82, 397 (March 1987), 171–185. doi:10.1080/01621459.1987.10478410 _eprint: https://www.tandfonline.com/doi/pdf/10.1080/01621459.1987.10478410.

[23] Molly Q Feldman and Carolyn Jane Anderson. 2024. Non-Expert Programmers in the Generative AI Future. In *Proceedings of the 3rd Annual Meeting of the Symposium on Human-Computer Interaction for Work* (Newcastle upon Tyne, United Kingdom) *(CHIWORK '24)*. Association for Computing Machinery, New York, NY, USA, Article 15, 19 pages. doi:10.1145/3663384.3663393

[24] Li Feng, Ryan Yen, Yuzhe You, Mingming Fan, Jian Zhao, and Zhicong Lu. 2024. CoPrompt: Supporting Prompt Sharing and Referring in Collaborative Natural Language Programming. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems (CHI '24)*. Association for Computing Machinery, New York, NY, USA, 1–21. doi:10.1145/3613904.3642212

[25] David M. Frohlic. 1993. The history and future of direct manipulation. *Behaviour & Information Technology* 12, 6 (Nov. 1993), 315–329. doi:10.1080/01449299308924396 _eprint: https://doi.org/10.1080/01449299308924396.

[26] Koumei Fukahori, Daisuke Sakamoto, Jun Kato, and Takeo Igarashi. 2014. CapStudio: An Interactive Screencast for Visual Application Development. In *CHI '14 Extended Abstracts on Human Factors in Computing Systems (CHI EA '14)*. Association for Computing Machinery, New York, NY, USA, 1453–1458. doi:10.1145/2559206.2581138

[27] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. 2023. Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 10932–10952. doi:10.18653/v1/2023.emnlp-main.674

[28] GitHub. 2026. GitHub Copilot. https://github.com/copilot/. Accessed: 2026-01-15.

[29] Major Greenwood. 1926. *The Natural Duration of Cancer.* Technical Report. London.

[30] The Open Group. 2026. diff. https://pubs.opengroup.org/onlinepubs/9799919799/. Accessed: 2026-02-22.

[31] Priyanshu Gupta, Avishree Khare, Yasharth Bajpai, Saikat Chakraborty, Sumit Gulwani, Aditya Kanade, Arjun Radhakrishna, Gustavo Soares, and Ashish Tiwari. 2023. Grace: Language Models Meet Code Edits. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (San Francisco, CA, USA) *(ESEC/FSE 2023)*. Association for Computing Machinery, New York, NY, USA, 1483–1495. doi:10.1145/3611643.3616253

[32] Rich Harris and Svelte Contributors. 2026. Svelte. https://svelte.dev/. Accessed: 2026-03-08.

[33] Sandra G. Hart. 2006. Nasa-Task Load Index (NASA-TLX); 20 Years Later. *Proceedings of the Human Factors and Ergonomics Society Annual Meeting* 50, 9 (Oct. 2006), 904–908. doi:10.1177/154193120605000909

[34] Sandra G. Hart and Lowell E. Staveland. 1988. Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical research. In *Human Mental Workload*. North-Holland, Oxford, England, 139–183. doi:10.1016/S0166-4115(08)62386-9

[35] Brian Hempel and Ravi Chugh. 2016. Semi-Automated SVG Programming via Direct Manipulation. In *Proceedings of the 29th Annual Symposium on User Interface Software and Technology (UIST '16)*. Association for Computing Machinery,

New York, NY, USA, 379–390. doi:10.1145/2984511.2984575

[36] Brian Hempel, Justin Lubin, and Ravi Chugh. 2019. Sketch-n-Sketch: Output-Directed Programming for SVG. In *Proceedings of the 32nd Annual ACM Symposium on User Interface Software and Technology (UIST '19)*. Association for Computing Machinery, New York, NY, USA, 281–292. doi:10.1145/3332165.3347925

[37] Keith C. Hendy, Kevin M. Hamilton, and Lois N. Landry. 1993. Measuring Subjective Workload: When Is One Scale Better Than Many? *Human Factors* 35, 4 (Dec. 1993), 579–601. doi:10.1177/001872089303500401

[38] Chris Hokamp and Qun Liu. 2017. Lexically Constrained Decoding for Sequence Generation Using Grid Beam Search. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Regina Barzilay and Min-Yen Kan (Eds.). Association for Computational Linguistics, Vancouver, Canada, 1535–1546. doi:10.18653/v1/P17-1141

[39] Edwin L. Hutchins, James D. Hollan, and Donald A. Norman. 1985. Direct Manipulation Interfaces. *Human–Computer Interaction* 1, 4 (Dec. 1985), 311–338. doi:10.1207/s15327051hci0104_2 _eprint: https://doi.org/10.1207/s15327051hci0104_2.

[40] StackBlitz Inc. 2026. StackBlitz. https://stackblitz.com/. Accessed: 2026-01-23.

[41] E. L. Kaplan and Paul Meier. 1958. Nonparametric Estimation from Incomplete Observations. *J. Amer. Statist. Assoc.* 53, 282 (1958), 457–481. doi:10.2307/2281868

[42] Majeed Kazemitabaar, Xinying Hou, Austin Henley, Barbara Jane Ericson, David Weintrop, and Tovi Grossman. 2024. How Novices Use LLM-based Code Generators to Solve CS1 Coding Tasks in a Self-Paced Learning Environment. In *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research (Koli Calling '23)*. Association for Computing Machinery, New York, NY, USA, 1–12. doi:10.1145/3631802.3631806

[43] Tae Soo Kim, DaEun Choi, Yoonseo Choi, and Juho Kim. 2022. Stylette: Styling the Web with Natural Language. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems (CHI '22)*. Association for Computing Machinery, New York, NY, USA, 1–17. doi:10.1145/3491102.3501931

[44] Sam Lau and Philip Guo. 2023. From "Ban It Till We Understand It" to "Resistance is Futile": How University Programming Instructors Plan to Adapt as More Students Use AI Code Generation and Explanation Tools such as ChatGPT and GitHub Copilot. In *Proceedings of the 2023 ACM Conference on International Computing Education Research - Volume 1 (ICER '23, Vol. 1)*. Association for Computing Machinery, New York, NY, USA, 106–121. doi:10.1145/3568813.3600138

[45] Toby Jia-Jun Li, Marissa Radensky, Justin Jia, Kirielle Singarajah, Tom M. Mitchell, and Brad A. Myers. 2019. PUMICE: A Multi-Modal Agent that Learns Concepts and Conditionals from Natural Language and Demonstrations. In *Proceedings of the 32nd Annual ACM Symposium on User Interface Software and Technology (UIST '19)*. Association for Computing Machinery, New York, NY, USA, 577–589. doi:10.1145/3332165.3347899

[46] Michael Xieyang Liu, Advait Sarkar, Carina Negreanu, Benjamin Zorn, Jack Williams, Neil Toronto, and Andrew D. Gordon. 2023. "What It Wants Me To Say": Bridging the Abstraction Gap Between End-User Programmers and Code-Generating Large Language Models. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (CHI '23)*. Association for Computing Machinery, New York, NY, USA, 1–31. doi:10.1145/3544548.3580817

[47] Francesca Lucchetti, Zixuan Wu, Arjun Guha, Molly Q Feldman, and Carolyn Jane Anderson. 2025. Substance Beats Style: Why Beginning Students Fail to Code with LLMs. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 8541–8610. doi:10.18653/v1/2025.naacl-long.433

[48] Damien Masson, Sylvain Malacria, Géry Casiez, and Daniel Vogel. 2024. DirectGPT: A Direct Manipulation Interface to Interact with Large Language Models. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems (CHI '24)*. Association for Computing Machinery, New York, NY, USA, 1–16. doi:10.1145/3613904.3642462

[49] Mikaël Mayer, Viktor Kuncak, and Ravi Chugh. 2018. Bidirectional Evaluation with Direct Manipulation. *Proceedings of the ACM on Programming Languages* 2, OOPSLA (Oct. 2018), 127:1–127:28. doi:10.1145/3276497

[50] Colin McDonnell. 2026. colinhacks/zod. https://github.com/colinhacks/zod original-date: 2020-03-07T20:59:08Z.

[51] Na Meng, Miryung Kim, and Kathryn S. McKinley. 2011. Systematic Editing: Generating Program Transformations from an Example. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation* (San Jose, California, USA) *(PLDI '11)*. Association for Computing Machinery, New York, NY, USA, 329–342. doi:10.1145/1993498.1993537

[52] Rada Mihalcea, Hugo Liu, and Henry Lieberman. 2006. NLP (Natural Language Processing) for NLP (Natural Language Programming). In *Computational Linguistics and Intelligent Text Processing*, Alexander Gelbukh (Ed.). Springer, Berlin, Heidelberg, 319–330. doi:10.1007/11671299_34

[53] David Moon, Andrew Blinn, and Cyrus Omar. 2023. Gradual Structure Editing with Obligations. In *2023 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 71–81. doi:10.1109/VL-HCC57772.2023.00016 ISSN: 1943-6106.

[54] Niels Mündler, Jingxuan He, Hao Wang, Koushik Sen, Dawn Song, and Martin Vechev. 2025. Type-Constrained Code Generation with Language Models. *Proc. ACM Program. Lang.* 9, PLDI, Article 171 (June 2025), 26 pages. doi:10.1145/3729274

[55] Vijayaraghavan Murali, Chandra Maddila, Imad Ahmad, Michael Bolin, Daniel Cheng, Negar Ghorbani, Renuka Fernandez, Nachiappan Nagappan, and Peter C. Rigby. 2024. AI-Assisted Code Authoring at Scale: Fine-Tuning, Deploying, and Mixed Methods Evaluation. *Proc. ACM Softw. Eng.* 1, FSE (July 2024), 48:1066–48:1085. doi:10.1145/3643774

[56] Shaan Nagy, Timothy Zhou, Nadia Polikarpova, and Loris D'Antoni. 2026. ChopChop: A Programmable Framework for Semantically Constraining the Output of Language Models. *Proc. ACM Program. Lang.* 10, POPL, Article 66 (Jan. 2026), 28 pages. doi:10.1145/3776708

[57] Sydney Nguyen, Hannah McLean Babe, Yangtian Zi, Arjun Guha, Carolyn Jane Anderson, and Molly Q Feldman. 2024. How Beginning Programmers and Code LLMs (Mis)read Each Other. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems (CHI '24)*. Association for Computing Machinery, New York, NY, USA, 1–26. doi:10.1145/3613904.3642706

[58] Gabrielle O'Brien. 2025. How Scientists Use Large Language Models to Program. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, 1–16. doi:10.1145/3706598.3713668

[59] Cyrus Omar, Ian Voysey, Ravi Chugh, and Matthew A. Hammer. 2019. Live Functional Programming with Typed Holes. In *Proceedings of the 46th ACM SIGPLAN Symposium on Principles of Programming Languages*, Vol. 3. Association for Computing Machinery, New York, NY, USA, Article 14, 32 pages. doi:10.1145/3290327

[60] Cyrus Omar, Ian Voysey, Michael Hilton, Jonathan Aldrich, and Matthew A. Hammer. 2017. Hazelnut: A Bidirectionally Typed Structure Editor Calculus. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL '17)*. Association for Computing Machinery, New York, NY, USA, 86–99. doi:10.1145/3009837.3009900

[61] William F. Opdyke. 1992. *Refactoring Object-Oriented Frameworks*. Ph. D. Dissertation. University of Illinois at Urbana-Champaign, USA.

[62] OpenAI. 2026. Codex. https://openai.com/codex/. Accessed: 2026-03-16.

[63] OpenAI. 2026. GPT-5. https://platform.openai.com/docs/models/gpt-5. Accessed: 2026-01-15.

[64] OpenAI. 2026. Structured model outputs | OpenAI API. https://developers.openai.com/api/docs/guides/structured-outputs/. Accessed: 2026-02-20.

[65] Tomas Petricek and Jonathan Edwards. 2025. Denicek: Computational Substrate for Document-Oriented End-User Programming. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology (UIST '25)*. Association for Computing Machinery, New York, NY, USA, 1–19. doi:10.1145/3746059.3747646

[66] Savvas Petridis, Michael Terry, and Carrie Jun Cai. 2023. PromptInfuser: Bringing User Interface Mock-ups to Life with Large Language Models. In *Extended Abstracts of the 2023 CHI Conference on Human Factors in Computing Systems (CHI EA '23)*. Association for Computing Machinery, New York, NY, USA, 1–6. doi:10.1145/3544549.3585628

[67] Savvas Petridis, Michael Terry, and Carrie J Cai. 2024. PromptInfuser: How Tightly Coupling AI and UI Design Impacts Designers' Workflows. In *Proceedings of the 2024 ACM Designing Interactive Systems Conference (DIS '24)*. Association for Computing Machinery, New York, NY, USA, 743–756. doi:10.1145/3643834.3661613

[68] Gabriel Poesia, Oleksandr Polozov, Vu Le, Ashish Tiwari, Gustavo Soares, Christopher Meek, and Sumit Gulwani. 2022. Synchromesh: Reliable Code Generation from Pre-Trained Language Models. doi:10.48550/arXiv.2201.11227 arXiv:2201.11227 [cs].

[69] Dorin Pomian, Abhiram Bellur, Malinda Dilhara, Zarina Kurbatova, Egor Bogomolov, Andrey Sokolov, Timofey Bryksin, and Danny Dig. 2024. EM-Assist: Safe Automated ExtractMethod Refactoring with LLMs. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering* (Porto de Galinhas, Brazil) *(FSE 2024)*. Association for Computing Machinery, New York, NY, USA, 582–586. doi:10.1145/3663529.3663803

[70] Shanto Rahman, Sachit Kuhar, Berk Cirisci, Pranav Garg, Shiqi Wang, Xiaofei Ma, Anoop Deoras, and Baishakhi Ray. 2025. UTFix: Change Aware Unit Test Repairing using LLM. *Proc. ACM Program. Lang.* 9, OOPSLA1, Article 85 (April 2025), 26 pages. doi:10.1145/3720419

[71] Donald B Roberts. 1999. *Practical Analysis for Refactoring*. Technical Report. USA.

[72] Advait Sarkar, Andy Gordon, Carina Negreanu, Christian Poelitz, Sruti Srinivasa Ragavan, and Ben Zorn. 2022. What is it like to program with artificial intelligence?. In *Proceedings of the 33rd Annual Conference of the Psychology of Programming Interest Group (PPIG 2022)*.

[73] JSON Schema. 2026. JSON Schema. https://json-schema.org/. Accessed: 2026-02-22.

[74] Vidya Setlur, Sarah E. Battersby, Melanie Tory, Rich Gossweiler, and Angel X. Chang. 2016. Eviza: A Natural Language Interface for Visual Analysis. In *Proceedings of the 29th Annual Symposium on User Interface Software and Technology (UIST '16)*. Association for Computing Machinery, New York, NY, USA, 365–377. doi:10.1145/2984511.2984588

[75] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. 2011. *A comprehensive study of Convergent and Commutative Replicated Data Types.* Research Report RR-7506. Inria – Centre Paris-Rocquencourt ; INRIA. 50 pages. https://inria.hal.science/inria-00555588

[76] Ben Shneiderman. 1983. Direct Manipulation: A Step Beyond Programming Languages. *Computer* 16, 8 (Aug. 1983), 57–69. doi:10.1109/MC.1983.1654471

[77] Ben Shneiderman and Pattie Maes. 1997. Direct Manipulation vs. Interface Agents. *Interactions* 4, 6 (Nov. 1997), 42–61. doi:10.1145/267505.267514

[78] Tim Teitelbaum and Thomas Reps. 1981. The Cornell Program Synthesizer: A Syntax-Directed Programming Environment. *Commun. ACM* 24, 9 (Sept. 1981), 563–573. doi:10.1145/358746.358755

[79] Sewen Thy, Andreea Costea, Kiran Gopinathan, and Ilya Sergey. 2023. Adventure of a Lifetime: Extract Method Refactoring for Rust. *Proc. ACM Program. Lang.* 7, OOPSLA2, Article 245 (Oct. 2023), 28 pages. doi:10.1145/3622821

[80] Priyan Vaithilingam, Elena L. Glassman, Jeevana Priya Inala, and Chenglong Wang. 2024. DynaVis: Dynamically Synthesized UI Widgets for Visualization Editing. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems (CHI '24)*. Association for Computing Machinery, New York, NY, USA, 1–17. doi:10.1145/3613904.3642639

[81] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems (CHI EA '22)*. Association for Computing Machinery, New York, NY, USA, 1–7. doi:10.1145/3491101.3519665

[82] Verbi. 2026. MaxQDA. https://maxqda.com/. Accessed: 2026-02-23.

[83] Vercel. 2026. Vercel V0. https://v0.app/. Accessed: 2026-03-02.

[84] Anqi Wang, Zhizhuo Yin, Yulu Hu, Yuanyuan Mao, Lei Han, Xin Tong, Keqing Jiao, and Pan Hui. 2025. Pinning "Reflection" on the Agenda: Investigating Reflection in Human–LLM Co-Creation for Creative Coding. In *Companion Publication of the 2025 Conference on Computer-Supported Cooperative Work and Social Computing (CSCW Companion '25)*. Association for Computing Machinery, New York, NY, USA, 249–255. doi:10.1145/3715070.3749234

[85] Yun Wang, Zhitao Hou, Leixian Shen, Tongshuang Wu, Jiaqi Wang, He Huang, Haidong Zhang, and Dongmei Zhang. 2023. Towards Natural Language-Based Visualization Authoring. *IEEE Transactions on Visualization and Computer Graphics* 29, 1 (Jan. 2023), 1222–1232. doi:10.1109/TVCG.2022.3209357

[86] Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. 2023. Copiloting the Copilots: Fusing Large Language Models with Completion Engines for Automated Program Repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (San Francisco, CA, USA) *(ESEC/FSE 2023)*. Association for Computing Machinery, New York, NY, USA, 172–184. doi:10.1145/3611643.3616271

[87] Linna Xie, Zhong Li, Yu Pei, Zhongzhen Wen, Kui Liu, Tian Zhang, and Xuandong Li. 2025. PReMM: LLM-Based Program Repair for Multi-method Bugs via Divide and Conquer. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 319 (Oct. 2025), 29 pages. doi:10.1145/3763097

[88] Hao Yan, Thomas D. Latoza, and Ziyu Yao. 2024. IntelliExplain: Enhancing Conversational Code Generation for Non-Professional Programmers. doi:10.48550/arXiv.2405.10250 arXiv:2405.10250 [cs].

[89] J.D. Zamfirescu-Pereira, Eunice Jun, Michael Terry, Qian Yang, and Bjoern Hartmann. 2025. Beyond Code Generation: LLM-supported Exploration of the Program Design Space. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, 1–17. doi:10.1145/3706598.3714154

[90] J.D. Zamfirescu-Pereira, Richmond Y. Wong, Bjoern Hartmann, and Qian Yang. 2023. Why Johnny Can't Prompt: How Non-AI Experts Try (and Fail) to Design LLM Prompts. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (CHI '23)*. Association for Computing Machinery, New York, NY, USA, 1–21. doi:10.1145/3544548.3581388

[91] Jialu Zhang, José Pablo Cambronero, Sumit Gulwani, Vu Le, Ruzica Piskac, Gustavo Soares, and Gust Verbruggen. 2024. PyDex: Repairing Bugs in Introductory Python Assignments using LLMs. *Proc. ACM Program. Lang.* 8, OOPSLA1, Article 133 (April 2024), 25 pages. doi:10.1145/3649850

[92] Xing Zhang, Guanchen Guo, Xiao He, and Zhenjiang Hu. 2023. Bidirectional Object-Oriented Programming: Towards Programmatic and Direct Manipulation of Objects. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (April 2023), 83:230–83:255. doi:10.1145/3586035

[93] Xing Zhang, Ruifeng Xie, Guanchen Guo, Xiao He, Tao Zan, and Zhenjiang Hu. 2024. Fusing Direct Manipulations into Functional Programs. *Proceedings of the ACM on Principles of Programming Languages* 8, POPL (Jan. 2024), 41:1211–41:1238. doi:10.1145/3632883

[94] Yangtian Zi, Luisa Li, Arjun Guha, Carolyn Anderson, and Molly Q Feldman. 2025. "I Would Have Written My Code Differently"': Beginners Struggle to Understand LLM-Generated Code. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25)*. Association for Computing Machinery,

New York, NY, USA, 1479–1488. doi:10.1145/3696630.3731663

[95] Albert Ziegler, Eirini Kalliamvakou, X. Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. 2024. Measuring GitHub Copilot's Impact on Productivity. *Commun. ACM* 67, 3 (Feb. 2024), 54–63. doi:10.1145/3633453

[96] Parker Ziegler, Justin Lubin, and Sarah E. Chasins. 2025. Fast Direct Manipulation Programming with Patch-Reconciliation Correspondence. *Proc. ACM Program. Lang.* 9, PLDI, Article 175 (June 2025), 26 pages. doi:10.1145/3729278

[97] Celal Ziftci, Stoyan Nikolov, Anna Sjövall, Bo Kim, Daniele Codecasa, and Max Kim. 2025. *Migrating Code At Scale With LLMs At Google.* Association for Computing Machinery, New York, NY, USA, 162–173. https://dl.acm.org/10.1145/3696630.3728542

A Diff$_{\mathcal{L}_{ck}}$ Encoding in cartokit$_{\text{DM+NL}}$ using Zod

$$
\begin{aligned}
\textbf{Program } P &::= \langle \textbf{camera} : \mathit{Cam}, \text{layers} : \{\text{id} \mapsto \ell\}\rangle \\
\text{Layers } \ell &::= \langle \text{type} : T, \text{data} : D, \text{style} : S\rangle \\
\text{Layer Types } T &::= \text{Point} \\
&\quad | \ \text{Line} \\
&\quad | \ \text{Polygon} \\
&\quad | \ \text{Choropleth} \\
&\quad | \ \text{Proportional Symbol} \\
&\quad | \ \text{Dot Density} \\
&\quad | \ \textbf{Heatmap} \\
\text{Data } D &::= [\text{d}_i, \ldots, \text{d}_N]^{N \geq 1} \\
\text{Channel } C &::= \text{fill-color} \mid \text{stroke-width} \mid \text{thresholds} \mid \text{classifier} \mid \text{min-radius} \mid \ldots \\
&\quad \textbf{+9 new channels} \\
\text{Style } S &::= \{C \mapsto \text{fn}\} \\
\textbf{Zoom } Z &::= r \quad r \in \mathbb{R},\ 0 \leq r \leq 24 \\
\textbf{Longitude } \mathit{Lng} &::= r \quad r \in \mathbb{R},\ -180 \leq r \leq 180 \\
\textbf{Latitude } \mathit{Lng} &::= r \quad r \in \mathbb{R},\ -90 \leq r \leq 90 \\
\textbf{Projection } \mathit{Pr} &::= \textbf{mercator} \mid \textbf{globe} \\
\textbf{Camera } \mathit{Cam} &::= \langle \textbf{projection} : \mathit{Pr}, \textbf{zoom} : Z, \textbf{center} : [\mathit{Lng}, \mathit{Lat}]\rangle \\
\\
\textbf{Maps } M &::= (\mathit{MS}, V) \\
\textbf{Mark Set } \mathit{MS} &::= \{m_1, \ldots, m_N\}^{N \geq 0} \\
\text{Mark } m &::= \texttt{mark}(\text{mt}, \text{id}, \text{d}, S) \\
\text{Mark type mt} &::= \text{Point} \mid \text{Line} \mid \text{Polygon} \\
\textbf{View } V &::= (\mathit{Pr}, Z, [\mathit{Lng}, \mathit{Lat}]) \\
\\
\text{Diff } \delta &::= \text{setChannel}(\text{id}, C, \text{fn}) \\
&\quad | \ \text{removeChannel}(\text{id}, C) \\
&\quad | \ \text{addLayer}(\text{id}, \ell) \\
&\quad | \ \text{removeLayer}(\text{id}) \\
&\quad | \ \text{transformLayer}(\text{id}, \text{transform}) \\
&\quad | \ \textbf{setCamera}(\mathit{Pr}, Z, [\mathit{Lng}, \mathit{Lat}]) \\
&\quad | \ \textbf{unknown}()
\end{aligned}
$$

Fig. 6. Relevant portions of the context-free grammar defining diff$_{\mathcal{L}_{ck}}$ from Ziegler et al. [96]. Additions for this work are **bolded**. fn is an abstract function that takes a layer datum d as input and returns a corresponding stylistic value for a given channel (C).

Recall that for this work, we largely reuse the instantiations of Diff$_{\mathcal{L}_{ck}}$ in `cartokit` described in §4 of Ziegler et al. [96], with some additions. We reproduce relevant sections of the context-free grammar for `diffs` in Figure 6, with additions for this work shown in **bold**.

As described in §3 and §4, we use constrained decoding in `cartokit`$_{\text{DM+NL}}$ to ensure an LLM produces valid `diffs` according to the above context-free grammar, and that `diffs` also adhere to static and dependent type constraints on their associated data. To demonstrate the specifics of how we implemented our constrained decoding framework, we describe the implementation of the setChannel(id, C, fn) `diff` as an illustrative example. setChannel takes a layer id, a channel (C) to add or modify on the associated layer's style (S) and a function (fn) mapping each datum (d) in the layer to a stylistic value for the supplied channel.

In the concrete implementation of `cartokit`$_{\text{DM+NL}}$, the setChannel `diff` is split out into cases based on the value of the channel C being set. For instance, Figure 7 shows the TypeScript definitions for the `StrokeWidthDiff` and `FillColorDiff` types, which instantiate the setChannel(id, C, fn) `diff` with C = fill-color and fn = d $\rightarrow$ `color`, and C = stroke-width and fn = d $\rightarrow$ `strokeWidth`, respectively.

```typescript
interface LayerDiff {
  layerId: CartoKitLayer['id'];
}

interface StrokeWidthDiff extends LayerDiff {
  type: 'stroke-width';
  payload: {
    strokeWidth: number;
  };
}

interface FillColorDiff extends LayerDiff {
  type: 'fill-color';
  payload: {
    color: string;
  };
}

// +42 additional cases...
```

Fig. 7. **TypeScript encodings of specific cases of the setChannel `diff` in `cartokit`$_{\text{DM+NL}}$'s implementation.** `cartokit`$_{\text{DM+NL}}$ (and `cartokit`) split setChannel into discrete cases based on the channel (C).

To leverage our constrained decoding framework, we translate these static types into JSON Schema [73] using the Zod schema validation library [50], which together allow us to encode additional dependent type constraints on the data associated with `diffs`. Figure 8 shows the Zod encodings of the `StrokeWidthDiff` and `FillColorDiff`. Notice that, beyond specifying static type constraints (e.g., `.number()` on `strokeWidth` and `.string()` on `color`), we are also able to specify dependent type constraints on the concrete values the LLM selects during `diff` generation (e.g., `.min(0)` on `strokeWidth` and `.regex(/^#[0-9A-Fa-f]{6}$/)` on `color`). These constraints help to ensure that natural language edits are semantically meaningful and, critically, do not violate application invariants (e.g., negative `stroke-width` values).

```
function StrokeWidthDiff(layerIdSchema: z.infer<typeof makeLayerIdSchema>) {
  return z.object({
    type: z.literal('stroke-width'),
    layerId: layerIdSchema,
    payload: z.object({
      strokeWidth: z.number().min(0)
    })
  });
}

function FillColorDiff(layerIdSchema: z.infer<typeof makeLayerIdSchema>) {
  return z.object({
    type: z.literal('fill-color'),
    layerId: layerIdSchema,
    payload: z.object({
      color: z.string().regex(/^#[0-9A-Fa-f]{6}$/)
    })
  });
}

// +43 additional cases... (42 + the Unknown diff)
```

Fig. 8. **Zod encodings of specific cases of the setChannel `diff` in $\texttt{cartokit}_{\text{DM+NL}}$'s implementation.** Zod supports specifying additional dependent type constraints on the values chosen by the LLM during `diff` generation.

We then take the discriminated union of all `diffs` (using `z.discriminatedUnion`) as our JSON Schema encoding of **Diff** from Figure 6. Finally, we wrap this discriminated union in a call to `z.array` to ensure the language model returns a (potentially empty) *sequence* of `diffs` per prompt. This encoding, along with the system prompt and user prompt, is sent to our language model (GPT-5 [63]). In response, the model returns a `diff` sequence that is subsequently applied by $\texttt{cartokit}_{\text{DM+NL}}$ using the same `patch-recon` architecture as direct manipulation edits. From the perspective of the `patch-recon` algorithm, the source of `diffs` (natural language versus direct manipulation) does not matter.

A.1 Additional Considerations

Referencing existing map layers. For a `diff` to be valid, not only must it contain all necessary fields (e.g., `type`, `layerId`, and `payload.strokeWidth`), but it also must refer to an existing layer id or, in the case of the addLayer `diff`, create a valid layer id. We enforce this constraint through a Zod schema that is dynamically generated at prompt time. Figure 9 shows the definition of `makeLayerIdSchema`, which constructs a union type that constrains generation to either an existing layer id or, when no layers are present on the map, a fresh id. This schema is passed into other `diff` schemas as the `layerIdSchema` argument (See Figure 8). We use a similar technique to encode other key application information, such as column names for the tabular data associated with layers.

```
function makeLayerIdSchema(layerIds: string[]) {
  const freshLayerIdSchema = z
    .string()
    .regex(/^[a-z0-9]+(?:-[a-z0-9]+)*__[a-z0-9]+$/);

  if (layerIds.length === 0) {
    return freshLayerIdSchema;
  }

  return z.union([
    z.literal(layerIds[0]),
    freshLayerIdSchema,
    ...layerIds.slice(1).map((id) => z.literal(id))
  ]);
}

```

Fig. 9. **Definition of `makeLayerIdSchema`.** `makeLayerIdSchema` runs dynamically at prompt time, generating a Zod schema that constrains generation to existing map layer ids or, in the case of the addLayer `diff`, a fresh layer id. Existing layer ids are sent from the client to the serverless function that invokes the OpenAI API.

*The **unknown** `diff`.* Not all user prompts can be mapped clearly to `diff` sequences. For example, users can (and participants did) prompt for functionality beyond the bounds of $\texttt{cartokit}_{\text{DM+NL}}$'s capabilities, such as requesting a complex spatial join between disparate layers. To handle these cases, we introduced the **unknown** `diff`. **unknown** is essentially a no-op; it has no impact on the program or output. The primary benefit is that it can be specifically handled by the client to signal to the user that their prompt did not have a clear mapping to available system edits.

B Extensions of patch-recon

As shown in Figure 6, this work extends the instantiation of patch-recon from Ziegler et al. [96] with several new `diffs`. We define the updated semantics for `patch` and `recon`, and extend the proof of patch-reconciliation correspondence below.

B.1 $\texttt{patch}_{\mathcal{L}_{ck}}$

B.1.1 Existing `diffs`. The semantics of existing $\texttt{patch}_{\mathcal{L}_{ck}}$ inference rules for addLayer, removeLayer, transformLayer, setChannel, and removeChannel are virtually unchanged. The only modification involves updating the **layers** property on the program (P) and leaving the existing definition of the **camera** property unmodified; these `diffs` have no impact on the definition of **camera**. We provide the updated case of Patch/SetChannel as an illustrative example in Figure 10; the other cases involve a similar adjustment.

B.1.2 New `diffs`. This work added two new `diffs`, **setCamera** and **unknown**, to accommodate programmer prompts that (i) change the map's camera in some way (e.g., "Display the map as a globe centered on Taiwan.") or (ii) request functionality that cannot currently be achieved through our existing `diffs` (e.g., "Run a spatial join between Layer A and Layer B using the within predicate."). Figure 11 defines `patch` for these `diffs`.

$$\textsc{Patch/Set-Channel}\quad \frac{P.\mathsf{layers}(\mathsf{id}) = \langle T, D, S\rangle}{\mathsf{patch}(\mathsf{setChannel}(\mathsf{id}, C, \mathsf{fn}), P) = [\mathsf{id} \mapsto \langle T, D, [C \mapsto \mathsf{fn}]S\rangle]P.\mathsf{layers}}$$

Fig. 10. An example modification of patch for existing diffs from Ziegler et al. [96] for $\mathcal{L}_{ck}$.

$$\textsc{Patch/Set-Camera}\quad \frac{}{\mathsf{patch}(\mathbf{setCamera}(Pr, Z, [Lng, Lat])) = [\mathsf{camera} \mapsto \langle Pr, Z, [Lng, Lat]\rangle]P} \qquad \textsc{Patch/Unknown}\quad \frac{}{\mathsf{patch}(\mathbf{unknown}()) = P}$$

Fig. 11. Our extension of patch for the **setCamera** and **unknown** diffs.

B.2 $\mathbf{recon}_{\mathcal{L}_{ck}}$

B.2.1 Existing diffs. The semantics of existing $\mathsf{recon}_{\mathcal{L}_{ck}}$ inference rules for addLayer, removeLayer, transformLayer, setChannel, and removeChannel are also virtually unchanged. The only modification here is that the output map (M) is modeled as a tuple containing a set of marks with an accompanying view triple (V). Existing diffs do not update the view during reconciliation. We provide the updated case of Recon/SetChannel as an illustrative example in Figure 12; the other cases involve a similar adjustment.

$$\textsc{Recon/Set-Channel}\quad \frac{\begin{array}{c} MS' = \{\mathbf{mark}(\mathsf{mt}, \mathsf{id}, \mathsf{d}, [C \mapsto \mathsf{fn}]S) \mid \mathbf{mark}(\mathsf{mt}, \mathsf{id}, \mathsf{d}, S) \in M(0)\} \\ MS'' = \{\mathbf{mark}(\mathsf{mt}, \mathsf{id}', \mathsf{d}, S) \mid \mathbf{mark}(\mathsf{mt}, \mathsf{id}', \mathsf{d}, S) \in M(0), \mathsf{id}' \neq \mathsf{id}\} \qquad V = M(1) \end{array}}{\mathsf{recon}(\mathsf{setChannel}(\mathsf{id}, C, \mathsf{fn}), M) = (MS' \cup MS'', V)}$$

Fig. 12. An example modification of recon for existing diffs from Ziegler et al. [96] for $\mathcal{L}_{ck}$.

B.2.2 New diffs. Figure 13 defines recon for the **setCamera** and **unknown** diffs.

$$\textsc{Recon/Set-Camera}\quad \frac{MS = M(0) \qquad V' = (Pr, Z, [Lng, Lat])}{\mathsf{recon}(\mathbf{setCamera}(Pr, Z, [Lng, Lat]), M) = (MS, V')}$$

$$\textsc{Recon/Unknown}\quad \frac{}{\mathsf{recon}(\mathbf{unknown}()) = (MS, V)}$$

Fig. 13. Our extension of recon for the **setCamera** and **unknown** diffs.

B.3 $\mathbf{eval}_{\mathcal{L}_{ck}}$

The definition of **eval** from Ziegler et al. [96] remains largely unchanged; we add an additional evalCamera rule that trivially evaluates a program's camera definition (*Cam*) to a view (V).

$$\textsc{Eval-Layer}\quad \frac{\mathsf{mt} = \mathsf{getMT}(T)}{\mathbf{evalLayer}(\mathsf{id}, \langle T, D, S\rangle) = \{\mathbf{mark}(\mathsf{mt}, \mathsf{id}, \mathsf{d}, S) \mid \mathsf{d} \in D\}}$$

$$\textsc{Eval-Camera}\quad \frac{}{\mathbf{evalCamera}(\langle Pr, Z, [Lng, Lat]\rangle) = (Pr, Z, [Lng, Lat])}$$

$$\textsc{Eval-Map}\quad \frac{}{\mathbf{eval}(P) = (\bigcup_{\mathsf{id} \in P.layers} \mathbf{evalLayer}(\mathsf{id}, P.layers[\mathsf{id}]), \mathbf{evalCamera}(P.camera))}$$

Fig. 14. Forward evaluation for our extension of $\mathcal{L}_{ck}$.

B.4 Proof of patch-reconciliation Correspondence

The proof of patch-reconciliation correspondence in Ziegler et al. [96] is a proof by cases on `diffs` (δ). Proofs for existing `diffs` are virtually unchanged; they just involve carrying the unmodified camera and view values through. Therefore, we elide them here, focusing on the two new cases.

Proof. Proceed by case analysis on δ.

(1) $\delta = \mathbf{setCamera}(Pr, Z, [Lng, Lat])$. Then

$$\begin{aligned}
\mathbf{eval}(\mathsf{patch}(\delta, P)) &= \mathbf{eval}([\mathsf{camera} \mapsto \langle Pr, Z, [Lng, Lat]\rangle]P) \\
&= (\bigcup_{\mathsf{id} \in P.layers} \mathbf{evalLayer}(\mathsf{id}, \langle T, D, S\rangle), \mathbf{evalCamera}(\langle Pr, Z, [Lng, Lat]\rangle)) \\
&= (MS, (Pr, Z, [Lng, Lat])) \\
&= \mathsf{recon}(\delta, \mathbf{eval}(P))
\end{aligned}$$

(2) $\delta = \mathbf{unknown}(())$. The proof is vacuously true as Patch-Unknown does not modify the program and Recon-Unknown does not modify the output.

□

Received 17 March 2025